Object-Oriented Programming in Java

Java Exercises

Solutions

Solution code for all 28 exercises, compiled and run on OpenJDK 25 (tests with JUnit 5); every output matches the sample run in the exercise sheet. From 2.7 on, every file is shown, followed by notes on the key decisions. At the end: the lines of the slides that do not compile, or are wrong, in Java 25.

Contents

Part 1 – Java environment **2**
1.1 Hello, command line . . . 2
1.2 Fix the conventions . . . 3
1.3 Jar file and magic number . . . 3

Part 2 – Java basics **3**
2.1 Calendar utilities . . . 3
2.2 Counter and Car . . . 5
2.3 Encapsulated bank account . . . 6
2.4 A shop in two packages . . . 7
2.5 Number tools . . . 9
2.6 Array tools . . . 10
2.7 A box office: static, final, factory methods, singleton . . . 11
2.8 Food delivery: enum, record and a checked exception . . . 15
2.9 A trail race: nested classes . . . 19

Part 3 – Characters and strings **23**
3.1 Job applications: text utilities . . . 23
3.2 A restaurant: formatted bills . . . 27
3.3 Supplier bids: StringBuilder, speed and the string pool . . . 31
3.4 Importing contacts: encodings . . . 34

Part 4 – JUnit 5 **37**
4.1 Exam-style tests for a parking lot . . . 37
4.2 Testing the food delivery of 2.8, then a suite . . . 39
4.3 Bug reproduction and TDD . . . 43

Part 5 – Inheritance, interfaces, lambdas **46**
5.1 Bank accounts: inheritance . . . 46
5.2 Customers and orders: equals, hashCode, toString, clone . . . 51
5.3 Bank operations: abstract classes and Template Method . . . 55
5.4 Pricing rules: interfaces, static and default methods . . . 60
5.5 Menu combos: Composite . . . 64
5.6 Restaurant chain reports: Comparable, Comparator, Strategy . . . 67
5.7 A train timetable: Iterable and Iterator . . . 71
5.8 Order statistics: functional interfaces, lambdas, method references . . . 75
5.9 Delivery notifications: Observer by composition (code + test) . . . 78

Slide errata **81**

Every solution below was compiled and run on OpenJDK 25 (tests with JUnit 5), and its output matches the sample run in the exercise. In 1.1–2.6 the `main` methods are in the exercises, so mostly only the classes you write are shown, several files in one code block, and obvious imports are left out. From 2.7 on, every file of the solution is shown on its own, imports included; only the `package` line is omitted, since the package is given at the top of each solution. Each solution ends with a few notes on its key decisions and traps. The full runnable project is in `java-exercises-src.zip`: one package per exercise under `src/`, tests under `test/`, and `run-all.sh` builds and runs everything.

Part 1 – Java environment

1.1 Hello, command line

```
public class Greeter {
    public static void main(String[] args) {
        if (args.length == 0) {
            String name = IO.readln("Name? ");          // console input (Java 25 IO class)
            if (name == null || name.trim().isEmpty()) {
                name = "world";
            }
            IO.println("Hello, " + name + "!");
        }
        for (String name : args) {
            IO.println("Hello, " + name + "!");
        }
    }
}
```

```
javac Greeter.java        # creates Greeter.class (bytecode)
java Greeter Ada Linus    # class name, without .class
```

Java 25 form – the parameter stays because the program uses the arguments (otherwise `void main()`):

```
class GreeterCompact {
    void main(String[] args) {
        if (args.length == 0) {
            String name = IO.readln("Name? ");
            if (name == null || name.trim().isEmpty()) {
                name = "world";
            }
            IO.println("Hello, " + name + "!");
        }
        for (String name : args) {
            IO.println("Hello, " + name + "!");
        }
    }
}
```

1.2 Fix the conventions

Saved as `BankAccount.java` (a public class lives in a file with exactly its name):

```java
public class BankAccount {
    static final double INTEREST_RATE = 0.02;   // constants: UPPER_CASE
    private double balance;                     // attributes, methods, parameters: lowerCamelCase
    public void depositMoney(double amount) { balance += amount; }
    public double getBalance() { return balance; }
}
```

1.3 Jar file and magic number

```
javac Greeter.java
printf 'Main-Class: Greeter\n' > manifest.txt     # the manifest must end with a newline
jar cvfm greeter.jar manifest.txt Greeter.class
jar tf greeter.jar                                 # META-INF/  META-INF/MANIFEST.MF  Greeter.class
java -jar greeter.jar Ada                          # Hello, Ada!
java -cp greeter.jar Greeter Ada                   # Hello, Ada!  (no manifest needed)
```

Without the final newline the `Main-Class` line is ignored and `java -jar` answers `no main manifest attribute`. Shortcut: `jar cvfe greeter.jar Greeter Greeter.class` sets the entry point without a manifest file.

```java
import java.nio.file.Files;
import java.nio.file.Path;

public class MagicNumber {
    public static void main(String[] args) throws Exception {
        byte[] data = Files.readAllBytes(Path.of(args[0]));
        String hex = "";
        for (int i = 0; i < 4 && i < data.length; i++) {
            // bytes are signed (-128..127): & 0xFF turns e.g. (byte)0xCA = -54 into 202
            hex += String.format("%02X", data[i] & 0xFF);
        }
        IO.println(args[0] + " starts with " + hex);
        if (hex.equals("CAFEBABE")) {
            IO.println("It is a Java class file");
        } else {
            IO.println("It is not a Java class file");
        }
    }
}
```

Part 2 – Java basics

2.1 Calendar utilities

```java
public class CalendarUtils {

    static final String[] MONTHS = {
        "january", "february", "march", "april", "may", "june",
        "july", "august", "september", "october", "november", "december"
    };
```

```
    public static String season(String month) {
        String result;
        switch (month.toLowerCase()) {
            case "december":
            case "january":
            case "february":
                result = "winter";
                break;
            case "march":
            case "april":
            case "may":
                result = "spring";
                break;
            case "june":
            case "july":
            case "august":
                result = "summer";
                break;
            case "september":
            case "october":
            case "november":
                result = "autumn";
                break;
            default:
                result = "unknown";
        }
        return result;
    }

    public static boolean isLeap(int year) {
        return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
    }

    public static int daysInMonth(String month, int year) {
        switch (month.toLowerCase()) {
            case "february":
                return isLeap(year) ? 29 : 28;
            case "april":
            case "june":
            case "september":
            case "november":
                return 30;
            case "january":
            case "march":
            case "may":
            case "july":
            case "august":
            case "october":
            case "december":
                return 31;
            default:
                return -1;
        }
    }

    public static int daysInYear(int year) {
        int total = 0;
        for (String m : MONTHS) {
            total += daysInMonth(m, year);
        }
        return total;
    }
```

```
    public static int nextLeapYear(int year) {
        int y = year + 1;
        while (!isLeap(y)) {
            y++;
        }
        return y;
    }

    public static void main(String[] args) {
        int[] years = {2023, 2024, 1900, 2000};
        if (args.length > 0) {
            years = new int[args.length];
            for (int i = 0; i < args.length; i++) {
                years[i] = Integer.parseInt(args[i]);
            }
        }
        for (int y : years) {
            String line = y + ": " + daysInYear(y) + " days";
            if (isLeap(y)) {
                line += " (leap)";
            }
            IO.println(line);
        }
        IO.println("Next leap year after 2025: " + nextLeapYear(2025));
        IO.println("JULY -> " + season("JULY") + ", Smarch -> " + season("Smarch"));
    }
}
```

2.2 Counter and Car

```
public class Counter {
    private int value;

    public Counter() { this(0); }                // delegates to Counter(int start)
    public Counter(int start) { value = start; }

    public Counter reset() { value = 0; return this; }
    public Counter increment(int by) { this.value += by; return this; }
    public Counter decrement(int by) { this.value -= by; return this; }
    public Counter print() { System.out.println(value); return this; }
    public int getValue() { return value; }
}

public class Car {
    private static int countBuiltCars = 0;

    private String brand;
    private String color;
    private boolean turnedOn;

    public Car() {
        this("Ferrari", "red");          // this(...) calls another constructor of Car
    }

    public Car(String brand) {
        this(brand, "white");
    }

    public Car(String brand, String color) {
        this.brand = brand;
```

```
        this.color = color;
        countBuiltCars++;                // every constructor ends up here
    }

    public Car(Car other) {              // copy constructor: a NEW, independent object
        this(other.brand, other.color);
        this.turnedOn = other.turnedOn;
    }

    public void turnOn() { turnedOn = true; }
    public void paint(String color) { this.color = color; }
    public void paint() { paint("white"); }      // overload: same name, other parameter list
    public boolean isOn() { return turnedOn; }
    public String getBrand() { return brand; }
    public String getColor() { return color; }

    public boolean sameAs(Car other) {   // contents, not identity
        return brand.equals(other.brand) && color.equals(other.color);
    }

    public static int getCountBuiltCars() { return countBuiltCars; }

    @Override
    public String toString() {
        return brand + " " + color + (turnedOn ? " (on)" : " (off)");
    }
}
```

`b = a` copies the reference (one object, two names), so painting `b` repaints `a`; `new Car(a)` makes a second object. The count is 5: `a`, `c`, the yellow Ferrari passed to `sameAs`, the Fiat and the Lancia.

2.3 Encapsulated bank account

```
public class BankAccount {
    public static final double MAX_WITHDRAWAL = 1000.0;
    private static int created = 0;

    private final int number;
    private final String owner;
    private double balance;

    public BankAccount(String owner) {
        this(owner, 0.0);
    }

    public BankAccount(String owner, double initialBalance) {
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Negative initial balance: " + initialBalance);
        }
        created++;                   // after the check: a rejected account uses no number
        this.number = created;
        this.owner = owner;
        this.balance = initialBalance;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Invalid amount: " + amount);
        }
        balance += amount;
```

```
    }

    public void withdraw(double amount) {
        if (amount <= 0 || amount > MAX_WITHDRAWAL || amount > balance) {
            throw new IllegalArgumentException("Invalid amount: " + amount);
        }
        balance -= amount;
    }

    public int getNumber() { return number; }
    public String getOwner() { return owner; }
    public double getBalance() { return balance; }
    public static int getCreatedAccounts() { return created; }

    @Override
    public String toString() {
        return "#" + number + " " + owner + ": " + balance;
    }
}
```

2.4 A shop in two packages

```
// src/it/polito/shop/Product.java
package it.polito.shop;

public class Product {
    private final String name;
    private final double price;
    int stock;                                      // package

    Product(String name, double price, int stock) {   // package: only Shop creates products
        this.name = name;
        this.price = price;
        this.stock = stock;
    }

    public String getName() { return name; }
    public double getPrice() { return price; }
    public int getStock() { return stock; }

    void removeStock(int qty) { stock -= qty; }  // package
}

// src/it/polito/shop/Discounts.java
package it.polito.shop;

class Discounts {                                   // not public: invisible outside the package
    static final int BULK_QUANTITY = 10;
    static final double BULK_FACTOR = 0.9;

    static double apply(double price, int qty) {
        double total = price * qty;
        if (qty >= BULK_QUANTITY) {
            total = total * BULK_FACTOR;
        }
        return total;
    }
}

// src/it/polito/shop/Shop.java
```

```java
package it.polito.shop;

public class Shop {
    private final Product[] products;
    private int count = 0;

    public Shop(int capacity) {
        products = new Product[capacity];
    }

    public boolean addProduct(String name, double price, int stock) {
        if (count == products.length || find(name) != null) {
            return false;
        }
        products[count++] = new Product(name, price, stock);
        return true;
    }

    public Product find(String name) {
        for (int i = 0; i < count; i++) {
            if (products[i].getName().equals(name)) {
                return products[i];
            }
        }
        return null;
    }

    public double sell(String name, int qty) {
        Product p = find(name);
        if (p == null || qty <= 0 || qty > p.stock) {
            return -1;
        }
        p.removeStock(qty);
        return Discounts.apply(p.getPrice(), qty);
    }

    public int stockOf(String name) {
        Product p = find(name);
        return p == null ? 0 : p.getStock();
    }
}

// src/it/polito/client/ShopApp.java
package it.polito.client;

import it.polito.shop.*;        // brings in the public classes only: Shop and Product

public class ShopApp {
    public static void main(String[] args) {
        // the main of the exercise
    }
}
```

The three forbidden lines fail to compile: `p.removeStock(1)` with "removeStock(int) is not public in Product; cannot be accessed from outside package"; `new Product("x", 1, 1)` with "Product(String,double,int) is not public in Product; ..."; `Discounts.apply(1.5, 10)` with "cannot find symbol", because `import it.polito.shop.*` brings in only the public classes (with the full name, `it.polito.shop.Discounts.apply(1.5, 10)`, the error is "Discounts is not public in it.polito.shop; cannot be accessed from outside package").

2.5 Number tools

```java
import java.util.Scanner;

public class NumberTools {

    public static int sumInts(String line) {
        Scanner s = new Scanner(line);
        int sum = 0;
        while (s.hasNext()) {
            if (s.hasNextInt()) {
                sum += s.nextInt();
            } else {
                s.next();               // skip the token that is not an int
            }
        }
        return sum;
    }

    public static String stats(String line) {
        Scanner s = new Scanner(line);
        int count = 0, sum = 0;
        int min = Integer.MAX_VALUE, max = Integer.MIN_VALUE;
        while (s.hasNext()) {
            if (s.hasNextInt()) {
                int v = s.nextInt();
                count++;
                sum += v;
                min = Math.min(min, v);
                max = Math.max(max, v);
            } else {
                s.next();
            }
        }
        if (count == 0) {
            return "count=0";
        }
        double avg = (double) sum / count;      // cast first, otherwise integer division
        return "count=" + count + " min=" + min + " max=" + max + " avg=" + avg;
    }

    public static String bases(int n) {
        return n + " = 0b" + Integer.toBinaryString(n) + " = 0x" + Integer.toHexString(n)
                + " = 0o" + Integer.toOctalString(n);
    }

    public static int percent(int part, int total) {
        return (int) Math.round(100.0 * part / total);   // Math.round(double) returns a long
    }

    public static boolean sameValue(Integer a, Integer b) {
        return a.equals(b);     // == compares objects: true only inside the cache -128..127
    }
}
```

2.6 Array tools

```java
import java.util.Arrays;

public class ArrayTools {

    public static int[][] pascal(int n) {
        int[][] t = new int[n][];
        for (int i = 0; i < n; i++) {
            t[i] = new int[i + 1];
            t[i][0] = 1;
            t[i][i] = 1;
            for (int j = 1; j < i; j++) {
                t[i][j] = t[i - 1][j - 1] + t[i - 1][j];
            }
        }
        return t;
    }

    public static int[][] transpose(int[][] m) {
        int[][] t = new int[m[0].length][m.length];
        for (int i = 0; i < m.length; i++) {
            for (int j = 0; j < m[i].length; j++) {
                t[j][i] = m[i][j];
            }
        }
        return t;
    }

    // O(1): rows are separate array objects, swapping the references is enough
    public static void swapRows(double[][] m, int i, int j) {
        double[] tmp = m[i];
        m[i] = m[j];
        m[j] = tmp;
    }

    public static double average(double... values) {
        double sum = 0;
        for (double v : values) {
            sum += v;
        }
        return sum / values.length;          // no arguments: 0.0 / 0 = NaN
    }

    public static int max(int first, int... others) {
        int res = first;
        for (int v : others) {
            if (v > res) res = v;
        }
        return res;
    }

    public static int[] concatSorted(int[] a, int[] b) {
        int[] r = Arrays.copyOf(a, a.length + b.length);
        for (int i = 0; i < b.length; i++) {
            r[a.length + i] = b[i];
        }
        Arrays.sort(r);
        return r;
    }
}
```

2.7 A box office: static, final, factory methods, singleton

Package ex2_7.

File AuditLog.java:

```java
// Exercise 2.7 R3 - lazy singleton: the only object is created at the first getInstance()
public class AuditLog {
    private static AuditLog instance;      // null until somebody asks for it

    static {                               // runs once, when the class is initialized
        IO.println("AuditLog class loaded");
    }

    private int count = 0;
    private String last = null;

    private AuditLog() {
        IO.println("AuditLog created");
    }

    public static AuditLog getInstance() {
        if (instance == null) {
            instance = new AuditLog();
        }
        return instance;
    }

    public void record(String message) {
        count++;
        last = message;
    }

    public int count() {
        return count;
    }

    public String last() {
        return last;
    }
}
```

File `Codes.java`:

```java
// Exercise 2.7 R2 - a utility class: no objects, only static methods
public final class Codes {

    private Codes() {              // nobody can write new Codes()
    }

    public static String code(char prefix, int n) {
        return String.format("%c%04d", prefix, n);
    }

    public static boolean isValid(String code) {
        if (code == null || code.length() != 5) {
            return false;
        }
        char first = code.charAt(0);
        if (first < 'A' || first > 'Z') {
            return false;
        }
        for (int i = 1; i < code.length(); i++) {
            char c = code.charAt(i);
            if (c < '0' || c > '9') {
                return false;
            }
        }
        return true;
    }

    public static int number(String code) {
        if (!isValid(code)) {
            throw new IllegalArgumentException("invalid code " + code);
        }
        return Integer.parseInt(code.substring(1));
    }
}
```

File `Money.java`:

```java
import java.util.Locale;

// Exercise 2.7 R1 - an immutable value object with static factory methods
public class Money {
    // built once, when the class is initialized; ofCents(0) always returns this object
    public static final Money ZERO = new Money(0);

    private final long cents;      // whole cents: no rounding errors when adding

    private Money(long cents) {    // private: objects are created only by the factories
        this.cents = cents;
    }

    public static Money ofCents(long cents) {
        if (cents < 0) {
            throw new IllegalArgumentException("negative amount " + cents);
        }
        return cents == 0 ? ZERO : new Money(cents);
    }

    public static Money of(double euros) {
        return ofCents(Math.round(euros * 100));
    }
```

```
    public Money plus(Money other) {
        return ofCents(cents + other.cents);     // a new object: this one never changes
    }

    public Money times(int qty) {
        return ofCents(cents * qty);
    }

    public long cents() {
        return cents;
    }

    @Override
    public String toString() {
        return String.format(Locale.US, "%d.%02d EUR", cents / 100, cents % 100);
    }
}
```

File `BoxOffice.java`:

```
import static ex2_7.Codes.*;      // code(...), isValid(...), number(...) without "Codes."

// Exercise 2.7 R4 - the facade
public class BoxOffice {
    public static final int MAX_SEATS = 500;
    private static final int MAX_EVENTS = 100;

    private static int officesCreated = 0;          // shared by all the offices

    // event i has the code E000(i+1): per-object arrays, so every office starts from E0001
    private final String[] names = new String[MAX_EVENTS];
    private final Money[] prices = new Money[MAX_EVENTS];
    private final int[] seatsLeft = new int[MAX_EVENTS];
    private int nEvents = 0;

    public BoxOffice() {
        officesCreated++;
    }

    public static int officesCreated() {
        return officesCreated;
    }

    public String addEvent(String name, int seats, Money price) {
        if (seats < 1 || seats > MAX_SEATS) {
            throw new IllegalArgumentException("invalid seats " + seats);
        }
        for (int i = 0; i < nEvents; i++) {
            if (names[i].equals(name)) {
                throw new IllegalArgumentException("duplicate event " + name);
            }
        }
        if (nEvents == MAX_EVENTS) {
            throw new IllegalStateException("full");
        }
        names[nEvents] = name;
        prices[nEvents] = price;
        seatsLeft[nEvents] = seats;
        nEvents++;
        return code('E', nEvents);
    }
```

```
    public Money sell(String code, int qty) {
        int i = indexOf(code);
        if (qty < 1) {
            throw new IllegalArgumentException("invalid quantity " + qty);
        }
        if (seatsLeft[i] < qty) {
            throw new IllegalArgumentException("not enough seats");
        }
        Money price = prices[i].times(qty);
        seatsLeft[i] -= qty;
        // the first call initializes AuditLog: only now "AuditLog class loaded" appears
        AuditLog.getInstance().record("sold " + qty + " x " + code);
        return price;
    }

    public int available(String code) {
        return seatsLeft[indexOf(code)];
    }

    private int indexOf(String code) {
        if (!isValid(code)) {
            throw new IllegalArgumentException("invalid code " + code);
        }
        int n = number(code);
        if (code.charAt(0) != 'E' || n < 1 || n > nEvents) {
            throw new IllegalArgumentException("unknown event " + code);
        }
        return n - 1;
    }
}
```

File `Main.java`:

```
public class Main {
    public static void main(String[] args) {
        IO.println(Money.of(12.5) + " | " + Money.ofCents(199).times(3) + " | "
                + Money.of(0.1).plus(Money.of(0.2)));
        IO.println((Money.of(0) == Money.ZERO) + " " + (Money.of(1) == Money.of(1)));
        IO.println(Codes.code('E', 7) + " " + Codes.isValid("E0007") + " " + Codes.isValid("e0007")
                + " " + Codes.number("E0042"));
        IO.println("start");
        BoxOffice turin = new BoxOffice();
        BoxOffice milan = new BoxOffice();
        String c1 = turin.addEvent("Concert", 100, Money.of(25));
        String c2 = turin.addEvent("Opera", 50, Money.of(40.5));
        String m1 = milan.addEvent("Concert", 80, Money.of(30));
        IO.println(c1 + " " + c2 + " " + m1 + " offices=" + BoxOffice.officesCreated());
        IO.println(turin.sell(c1, 3) + " left=" + turin.available(c1));
        IO.println(milan.sell(m1, 2) + " left=" + milan.available(m1));
        try {
            turin.sell("E0099", 1);
        } catch (IllegalArgumentException e) {
            IO.println("error: " + e.getMessage());
        }
        try {
            turin.addEvent("Opera", 10, Money.ZERO);
        } catch (IllegalArgumentException e) {
            IO.println("error: " + e.getMessage());
        }
        IO.println(AuditLog.getInstance().count() + " " + AuditLog.getInstance().last());
    }
}
```

`Money` keeps a `long` of cents and has a private constructor, so every object comes from `ofCents`, which rejects negative values and returns the one `ZERO` for 0: that is why `Money.of(0) == Money.ZERO` is true while two `Money.of(1)` are different objects (`==` is safe only for that shared constant). `of` rounds once with `Math.round(euros * 100)`, so 0.1 + 0.2 is exactly 30 cents, and `toString` prints the cents as integers (`cents / 100`, then `cents % 100` with `%02d`), with no `double` involved. `AuditLog`'s static block runs when the class is initialized, at its first use, which is the first `sell` and not `new BoxOffice()`, so `BoxOffice` must not keep the log in a static field or the two lines would print too early. The event arrays and their counter are instance fields, so every office starts from `E0001`, while `officesCreated` is static; the number in a code is the array index plus one, so `sell` throws `invalid code` for a malformed code and `unknown event` for a well-formed one that is out of range or has a prefix other than `E`, and it computes the price before it takes the seats. The text gives no message for the seats check or for a quantity below 1, so they are `invalid seats <n>` and `invalid quantity <n>`.

2.8 Food delivery: enum, record and a checked exception

Package `ex2_8`.

File `DeliveryException.java`:

```
public class DeliveryException extends Exception {
    public DeliveryException(String message) {
        super(message);
    }
}
```

File `Category.java`:

```
public enum Category {
    STARTER, MAIN_COURSE, DESSERT, DRINK;

    public String label() {
        return name().replace('_', ' ');     // MAIN_COURSE -> MAIN COURSE
    }
}
```

File `Line.java`:

```
public record Line(MenuItem item, int qty) {
}
```

File `MenuItem.java`:

```
import java.util.Locale;

public record MenuItem(String description, Category category, double price, int prepTime) {

    @Override
    public String toString() {
        // Locale.US: always a decimal point, whatever the language of the computer
        return String.format(Locale.US, "[%s] %s : %.2f", category.label(), description, price);
    }
}
```

File `Status.java`:

```
public enum Status {
    NEW, CONFIRMED, PREPARATION, ON_DELIVERY, DELIVERED;

    public Status next() {
        Status[] all = values();             // the constants in declaration order
        if (ordinal() == all.length - 1) {
            throw new IllegalStateException("no status after " + this);
        }
        return all[ordinal() + 1];
    }
}
```

File `Delivery.java`:

```
// Exercise 2.8 - the facade: enums, records and a checked exception
public class Delivery {
    private static final int MAX_ITEMS = 50;
    private static final int MAX_ORDERS = 100;
    private static final int MAX_LINES = 10;

    private final MenuItem[] items = new MenuItem[MAX_ITEMS];
    private int nItems = 0;

    // order with id k is at index k-1; the counter is NOT static, so every Delivery starts from 1
    private final Line[][] lines = new Line[MAX_ORDERS][MAX_LINES];
    private final int[] nLines = new int[MAX_ORDERS];
    private final Status[] status = new Status[MAX_ORDERS];
    private int nOrders = 0;

    // ----- R2 menu
```

```
    public void addItem(String description, Category category, double price, int prepTime)
            throws DeliveryException {
        for (int i = 0; i < nItems; i++) {
            if (items[i].description().equals(description)) {
                throw new DeliveryException("duplicate item " + description);
            }
        }
        if (nItems == MAX_ITEMS) {
            throw new IllegalStateException("full");
        }
        items[nItems++] = new MenuItem(description, category, price, prepTime);
    }

    public MenuItem findItem(String pattern) throws DeliveryException {
        String lower = pattern.toLowerCase();
        MenuItem found = null;
        int matches = 0;
        for (int i = 0; i < nItems; i++) {
            if (items[i].description().toLowerCase().contains(lower)) {
                found = items[i];
                matches++;
            }
        }
        if (matches == 0) {
            throw new DeliveryException("no item matches " + pattern);
        }
        if (matches > 1) {
            throw new DeliveryException(matches + " items match " + pattern);
        }
        return found;
    }

    // ----- R3 orders

    public int newOrder() {
        if (nOrders == MAX_ORDERS) {
            throw new IllegalStateException("full");
        }
        status[nOrders] = Status.NEW;
        nOrders++;
        return nOrders;
    }

    public int addToOrder(int orderId, String pattern, int qty) throws DeliveryException {
        int o = index(orderId);
        if (status[o] != Status.NEW) {
            throw new DeliveryException("order " + orderId + " is " + status[o]);
        }
        if (qty < 1) {
            throw new DeliveryException("invalid quantity " + qty);
        }
        MenuItem item = findItem(pattern);
        for (int j = 0; j < nLines[o]; j++) {
            if (lines[o][j].item().equals(item)) {
                // a record cannot change: replace the line with a new one
                lines[o][j] = new Line(item, lines[o][j].qty() + qty);
                return lines[o][j].qty();
            }
        }
        if (nLines[o] == MAX_LINES) {
            throw new IllegalStateException("full");
        }
        lines[o][nLines[o]++] = new Line(item, qty);
```

```
        return qty;
    }

    public double total(int orderId) throws DeliveryException {
        int o = index(orderId);
        double total = 0;
        for (int j = 0; j < nLines[o]; j++) {
            total += lines[o][j].item().price() * lines[o][j].qty();
        }
        return total;
    }

    public String showOrder(int orderId) throws DeliveryException {
        int o = index(orderId);
        String[] parts = new String[nLines[o]];
        for (int j = 0; j < nLines[o]; j++) {
            parts[j] = lines[o][j].item().description() + ", " + lines[o][j].qty();
        }
        return String.join("; ", parts);
    }

    // ----- R4 states

    public Status status(int orderId) throws DeliveryException {
        return status[index(orderId)];
    }

    public int confirm(int orderId) throws DeliveryException {
        return move(orderId, Status.NEW);
    }

    public int start(int orderId) throws DeliveryException {
        return move(orderId, Status.CONFIRMED);
    }

    public int deliver(int orderId) throws DeliveryException {
        return move(orderId, Status.PREPARATION);
    }

    public int complete(int orderId) throws DeliveryException {
        return move(orderId, Status.ON_DELIVERY);
    }

    // the four moves differ only in the state they start from
    private int move(int orderId, Status expected) throws DeliveryException {
        int o = index(orderId);
        if (status[o] != expected) {
            throw new DeliveryException("order " + orderId + " is " + status[o]
                    + ", expected " + expected);
        }
        status[o] = expected.next();
        return switch (status[o]) {
            case CONFIRMED -> 5 + maxPrepTime(o) + 15;
            case PREPARATION -> maxPrepTime(o) + 15;
            case ON_DELIVERY -> 15;
            case DELIVERED -> 0;
            case NEW -> throw new IllegalStateException("no move leads to NEW");
        };
    }

    private int maxPrepTime(int o) {
        int max = 0;
        for (int j = 0; j < nLines[o]; j++) {
```

```
            max = Math.max(max, lines[o][j].item().prepTime());
        }
        return max;
    }

    private int index(int orderId) throws DeliveryException {
        if (orderId < 1 || orderId > nOrders) {
            throw new DeliveryException("unknown order " + orderId);
        }
        return orderId - 1;
    }
}
```

File `Main.java`:

```
public class Main {
    public static void main(String[] args) throws DeliveryException {
        Delivery d = new Delivery();
        d.addItem("Chinese noodles", Category.MAIN_COURSE, 8.5, 20);
        d.addItem("Spring rolls", Category.STARTER, 4.0, 10);
        d.addItem("Green tea", Category.DRINK, 2.5, 2);
        d.addItem("Mochi", Category.DESSERT, 3.0, 5);
        IO.println(d.findItem("NOODLE") + " | " + d.findItem("tea"));
        try {
            d.findItem("n");
        } catch (DeliveryException e) {
            IO.println("error: " + e.getMessage());
        }
        int o1 = d.newOrder(), o2 = d.newOrder();
        d.addToOrder(o1, "noodles", 2);
        d.addToOrder(o1, "rolls", 1);
        IO.println(d.addToOrder(o1, "noodles", 1) + " " + d.showOrder(o1)
                + " total=" + d.total(o1));
        IO.println(d.status(o1) + " " + d.confirm(o1) + " " + d.start(o1) + " " + d.deliver(o1)
                + " " + d.complete(o1) + " " + d.status(o1));
        try {
            d.start(o2);
        } catch (DeliveryException e) {
            IO.println("error: " + e.getMessage());
        }
        IO.println(Status.NEW.next() + " " + Category.MAIN_COURSE.label() + " "
                + Status.valueOf("ON_DELIVERY").ordinal());
    }
}
```

`label()` is `name().replace('_', ' ')`, `next()` is `values()[ordinal() + 1]` after checking for the last constant, and `MenuItem` overrides only `toString()`, with `Locale.US` so the price always has a decimal point. A `Line` cannot change, so adding an item that is already in the order replaces the line with `new Line(item, old + qty)`; `addToOrder` checks the order, its state, the quantity and the item before it touches anything, so a failed call changes nothing. The four moves call one private `move(id, expected)`, which checks the current state, sets `expected.next()` and computes the minutes with a switch expression on the new state; the switch must be exhaustive, so `NEW` gets a case that throws. The order counter is an instance field: a `static` one would make the ids continue from one `Delivery` to the next and break the tests of 4.2. Duplicate descriptions are compared with `equals` (case-sensitive), with the unspecified message `duplicate item <description>`.

2.9 A trail race: nested classes

Package `ex2_9`.

File `TrailException.java`:

```java
public class TrailException extends Exception {
    public TrailException(String message) {
        super(message);
    }
}
```

File `Race.java`:

```java
// Exercise 2.9 - static nested, inner and local classes
public class Race {
    private static final int MAX_RUNNERS = 100;
    private static final int MAX_CHECKPOINTS = 20;

    private final Runner[] runners = new Runner[MAX_RUNNERS];
    private int nRunners = 0;
    private final Checkpoint[] checkpoints = new Checkpoint[MAX_CHECKPOINTS];
    private int nCheckpoints = 0;

    // ----- R1 runners

    // static nested: a Runner does not need a Race object to exist
    public static class Runner {
        private final int bib;
        private final String name;
        private final String surname;

        private Runner(int bib, String name, String surname) {   // only Race creates runners
            this.bib = bib;
            this.name = name;
            this.surname = surname;
        }

        public int bib() {
            return bib;
        }

        public String name() {
            return name;
        }

        public String surname() {
            return surname;
        }

        @Override
        public String toString() {
            return bib + " " + surname + " " + name;
        }
    }

    public int newRunner(String name, String surname) {
        if (nRunners == MAX_RUNNERS) {
            throw new IllegalStateException("full");
        }
        runners[nRunners] = new Runner(nRunners + 1, name, surname);
        nRunners++;
        return nRunners;
    }

    public Runner getRunner(int bib) throws TrailException {
        if (bib < 1 || bib > nRunners) {
```

```
            throw new TrailException("unknown bib " + bib);
        }
        return runners[bib - 1];
    }

    // ----- R2 checkpoints

    // inner (not static): every Checkpoint is linked to the Race that created it
    public class Checkpoint {
        private final String name;
        private final int position;
        private Passage head = null;          // passages sorted by minute

        private Checkpoint(String name, int position) {
            this.name = name;
            this.position = position;
        }

        public String name() {
            return name;
        }

        public int position() {
            return position;
        }

        // ----- R3 passages

        public void record(int bib, int minute) throws TrailException {
            // getRunner is the method of the enclosing Race (Race.this.getRunner)
            Runner runner = getRunner(bib);
            for (Passage p = head; p != null; p = p.next) {
                if (p.runner == runner) {
                    throw new TrailException(bib + " already passed " + name);
                }
            }
            Passage passage = new Passage(runner, minute);
            if (head == null || minute < head.minute) {
                passage.next = head;
                head = passage;
                return;
            }
            // stop after the last passage with minute <= the new one:
            // equal minutes keep the arrival order
            Passage prev = head;
            while (prev.next != null && prev.next.minute <= minute) {
                prev = prev.next;
            }
            passage.next = prev.next;
            prev.next = passage;
        }

        public String ranking() {
            String separator = ", ";
            // local class: it can use the local variable separator (effectively final)
            class Joiner {
                private String text = "";

                void add(String part) {
                    text = text.isEmpty() ? part : text + separator + part;
                }

                String result() {
```

```
                    return text;
                }
            }
            Joiner joiner = new Joiner();
            for (Passage p = head; p != null; p = p.next) {
                joiner.add(p.runner + " (" + p.minute + ")");
            }
            return joiner.result();
        }
    }

    // private static nested: a node of the list, an implementation detail nobody else sees
    private static class Passage {
        Runner runner;
        int minute;
        Passage next;

        Passage(Runner runner, int minute) {
            this.runner = runner;
            this.minute = minute;
        }
    }

    public Checkpoint addCheckpoint(String name) throws TrailException {
        for (int i = 0; i < nCheckpoints; i++) {
            if (checkpoints[i].name.equals(name)) {
                throw new TrailException("duplicate checkpoint " + name);
            }
        }
        if (nCheckpoints == MAX_CHECKPOINTS) {
            throw new IllegalStateException("full");
        }
        Checkpoint c = new Checkpoint(name, nCheckpoints);   // same as this.new Checkpoint(...)
        checkpoints[nCheckpoints++] = c;
        return c;
    }

    public Checkpoint getCheckpoint(String name) throws TrailException {
        for (int i = 0; i < nCheckpoints; i++) {
            if (checkpoints[i].name.equals(name)) {
                return checkpoints[i];
            }
        }
        throw new TrailException("unknown checkpoint " + name);
    }

    // ----- R4

    public Runner leader() {
        // from the furthest checkpoint back: the first one with passages decides
        for (int i = nCheckpoints - 1; i >= 0; i--) {
            if (checkpoints[i].head != null) {
                return checkpoints[i].head.runner;     // the earliest minute there
            }
        }
        return null;
    }
}
```

File `Main.java`:

```
public class Main {
    public static void main(String[] args) throws TrailException {
        Race race = new Race();
        int rossi = race.newRunner("Mario", "Rossi");
        int verdi = race.newRunner("Luca", "Verdi");
        int bianchi = race.newRunner("Anna", "Bianchi");
        Race.Checkpoint start = race.addCheckpoint("Courmayeur");
        Race.Checkpoint arnad = race.addCheckpoint("Arnad");
        race.addCheckpoint("La Thuile");
        IO.println(race.getRunner(bianchi) + " | " + arnad.name() + " " + arnad.position());
        start.record(rossi, 0);
        start.record(verdi, 0);
        start.record(bianchi, 0);
        arnad.record(rossi, 102);
        arnad.record(bianchi, 95);
        IO.println(start.ranking());
        IO.println(arnad.ranking());
        IO.println("leader: " + race.leader());
        race.getCheckpoint("La Thuile").record(verdi, 300);
        IO.println("leader: " + race.leader());
        try {
            arnad.record(bianchi, 120);
        } catch (TrailException e) {
            IO.println("error: " + e.getMessage());
        }
        try {
            arnad.record(9, 130);
        } catch (TrailException e) {
            IO.println("error: " + e.getMessage());
        }
    }
}
```

`Runner` is a static nested class because a runner does not need a race object, and its private constructor means only `Race` creates runners; `Checkpoint` is an inner class, so its `record` calls `getRunner` of the race that created it (`Race.this.getRunner`), and a checkpoint of another race sees another list of runners. `Passage` is a private static node class, and `record` inserts after the last node whose minute is `<=` the new one, which keeps arrival order for equal minutes (with `<` ties would come out reversed). The local class `Joiner` can read `separator` only because the variable is effectively final. `leader()` scans the checkpoints from the last one back, and the head of the first non-empty list is the earliest runner at the furthest checkpoint. The messages for a duplicate or unknown checkpoint are not given; they are `duplicate checkpoint <name>` and `unknown checkpoint <name>`.

Part 3 – Characters and strings

3.1 Job applications: text utilities

Package `ex3_1`.

File `ApplicationException.java`:

```java
public class ApplicationException extends Exception {
    public ApplicationException(String message) {
        super(message);
    }
}
```

File `Applications.java`:

```java
import java.util.Arrays;

// Exercise 3.1 - text utilities: trim, case, split, parseInt, join
public class Applications {
    private static final int MAX_SKILLS = 50;
    private static final int MAX_APPLICANTS = 50;
    private static final int MAX_CAPABILITIES = 10;

    private final String[] skills = new String[MAX_SKILLS];
    private int nSkills = 0;

    // applicant i has the skills capSkills[i] (sorted) with the levels capLevels[i]
    private final String[] names = new String[MAX_APPLICANTS];
    private final String[][] capSkills = new String[MAX_APPLICANTS][];
    private final int[][] capLevels = new int[MAX_APPLICANTS][];
    private int nApplicants = 0;

    // ----- R1 skills

    public void addSkills(String... newSkills) throws ApplicationException {
        // check everything first, store afterwards: an exception leaves the data unchanged
        String[] normalized = new String[newSkills.length];
        for (int i = 0; i < newSkills.length; i++) {
            String skill = newSkills[i].trim().toLowerCase();
            if (indexOf(skills, nSkills, skill) >= 0 || indexOf(normalized, i, skill) >= 0) {
                throw new ApplicationException("duplicate skill " + skill);
            }
            normalized[i] = skill;
        }
        if (nSkills + normalized.length > MAX_SKILLS) {
            throw new IllegalStateException("full");
        }
        for (String skill : normalized) {
            skills[nSkills++] = skill;
        }
    }

    // ----- R2 applicants

    public void addApplicant(String name, String capabilities) throws ApplicationException {
        String normalized = normalizeName(name);
        if (indexOf(names, nApplicants, normalized) >= 0) {
            throw new ApplicationException("duplicate applicant " + normalized);
        }
        // parse into local arrays, kept sorted by skill; the applicant is stored only at the end
        String[] skillsOf = new String[MAX_CAPABILITIES];
        int[] levelsOf = new int[MAX_CAPABILITIES];
        int n = 0;
        if (!capabilities.isBlank()) {
            // -1 here too: in "java:9," the empty last part is an invalid capability
            for (String part : capabilities.split(",", -1)) {
                // limit -1 keeps empty pieces, so the length is always (number of ':') + 1
                String[] pieces = part.split(":", -1);
```

```
                if (pieces.length != 2) {
                    throw new ApplicationException("invalid capability " + part.trim());
                }
                String skill = pieces[0].trim().toLowerCase();
                if (indexOf(skills, nSkills, skill) < 0) {
                    throw new ApplicationException("unknown skill " + skill);
                }
                if (indexOf(skillsOf, n, skill) >= 0) {
                    throw new ApplicationException("duplicate skill " + skill);
                }
                int level = parseLevel(pieces[1].trim());
                if (n == MAX_CAPABILITIES) {
                    throw new IllegalStateException("full");
                }
                // insertion sort: shift the bigger skills one place to the right
                int k = n;
                while (k > 0 && skillsOf[k - 1].compareTo(skill) > 0) {
                    skillsOf[k] = skillsOf[k - 1];
                    levelsOf[k] = levelsOf[k - 1];
                    k--;
                }
                skillsOf[k] = skill;
                levelsOf[k] = level;
                n++;
            }
        }
        if (nApplicants == MAX_APPLICANTS) {
            throw new IllegalStateException("full");
        }
        names[nApplicants] = normalized;
        capSkills[nApplicants] = Arrays.copyOf(skillsOf, n);
        capLevels[nApplicants] = Arrays.copyOf(levelsOf, n);
        nApplicants++;
    }

    // ----- R3 queries

    public String getCapabilities(String name) throws ApplicationException {
        int i = applicant(name);
        String[] parts = new String[capSkills[i].length];
        for (int k = 0; k < parts.length; k++) {
            parts[k] = capSkills[i][k] + ":" + capLevels[i][k];
        }
        return String.join(",", parts);
    }

    public int level(String name, String skill) throws ApplicationException {
        int i = applicant(name);
        int k = indexOf(capSkills[i], capSkills[i].length, skill.trim().toLowerCase());
        return k < 0 ? 0 : capLevels[i][k];
    }

    public String findApplicants(String pattern) {
        String lower = pattern.toLowerCase();
        String[] found = new String[nApplicants];
        int n = 0;
        for (int i = 0; i < nApplicants; i++) {
            if (names[i].toLowerCase().contains(lower)) {
                found[n++] = names[i];
            }
        }
        String[] result = Arrays.copyOf(found, n);
        Arrays.sort(result);
```

```
        return String.join(", ", result);
    }

    // ----- helpers

    // "  mario   ROSSI " -> "Mario Rossi"
    private static String normalizeName(String name) {
        String[] words = name.trim().split("\\s+");      // \s+ = one or more white spaces
        for (int i = 0; i < words.length; i++) {
            String w = words[i];
            if (!w.isEmpty()) {                           // only for an empty name
                words[i] = w.substring(0, 1).toUpperCase() + w.substring(1).toLowerCase();
            }
        }
        return String.join(" ", words);
    }

    private static int parseLevel(String text) throws ApplicationException {
        int level;
        try {
            level = Integer.parseInt(text);
        } catch (NumberFormatException e) {
            throw new ApplicationException("invalid level " + text);
        }
        if (level < 1 || level > 10) {
            throw new ApplicationException("invalid level " + text);
        }
        return level;
    }

    private int applicant(String name) throws ApplicationException {
        String normalized = normalizeName(name);
        int i = indexOf(names, nApplicants, normalized);
        if (i < 0) {
            throw new ApplicationException("unknown applicant " + normalized);
        }
        return i;
    }

    // position of value among the first n elements of array, -1 if absent (equals, never ==)
    private static int indexOf(String[] array, int n, String value) {
        for (int i = 0; i < n; i++) {
            if (array[i].equals(value)) {
                return i;
            }
        }
        return -1;
    }
}
```

File `Main.java`:

```
public class Main {
    public static void main(String[] args) throws ApplicationException {
        Applications a = new Applications();
        a.addSkills("Java", " SQL ", "uml");
        a.addApplicant("  mario   ROSSI ", "sql:7, java : 9");
        a.addApplicant("anna bianchi", "uml:10");
        a.addApplicant("Marta Rossini", "");
        IO.println(a.getCapabilities("Mario Rossi") + " | " + a.getCapabilities("ANNA BIANCHI")
                + " | [" + a.getCapabilities("marta rossini") + "]");
        IO.println(a.findApplicants("ross") + " | " + a.findApplicants("AN"));
        IO.println(a.level("Mario Rossi", "java") + " " + a.level("Anna Bianchi", "java"));
        String[] bad = { "java:11", "cobol:5", "java=5", "java:x" };
        for (String caps : bad) {
            try {
                a.addApplicant("Test Person", caps);
            } catch (ApplicationException e) {
                IO.println("error: " + e.getMessage());
            }
        }
        try {
            a.addSkills("python", "JAVA");
        } catch (ApplicationException e) {
            IO.println("error: " + e.getMessage());
        }
    }
}
```

Every method that can throw first validates into local arrays and only then copies into the fields, so a failed `addSkills` adds none of its skills and a failed `addApplicant` stores nothing; duplicates within one `addSkills` call are found by searching the part of the local array already filled. Names are normalized with `trim().split("\\s+")`, capitalizing each word and joining with `String.join(" ", ...)`, and always compared with `equals` after normalization; skills are trimmed and lower-cased on every path (storing, parsing a capability, `level`). Each capability is split with `split(":", -1)`, whose length is always the number of colons plus one, so `"java:"` is an `invalid level` (empty text) rather than being silently cut to one piece; the commas are split with `-1` too, so an empty part, as in `"java:9,"` or `"java:9,,sql:7"`, is always an `invalid capability`. The capabilities are kept sorted by skill with an insertion step; sorting the `"skill:level"` strings would be wrong for skills like `c` and `c++`, because `'+'` comes before `':'`. Two cases are not in the text: a skill repeated in one capabilities string is rejected as `duplicate skill <skill>`, and `level` throws `unknown applicant <name>` like `getCapabilities`.

3.2 A restaurant: formatted bills

Package `ex3_2`.

File `RestaurantException.java`:

```
public class RestaurantException extends Exception {
    public RestaurantException(String message) {
        super(message);
    }
}
```

File `Restaurant.java`:

```java
import java.util.Arrays;
import java.util.Locale;

// Exercise 3.2 - formatted text with String.format
public class Restaurant {
    private static final int MAX_MENUS = 50;
    private static final int MAX_GROUPS = 50;
    private static final int SEATS_PER_TABLE = 4;

    private final String name;
    private int freeTables;
    private int refused = 0;
    private double income = 0;

    private final String[] menuNames = new String[MAX_MENUS];
    private final double[] menuPrices = new double[MAX_MENUS];
    private int nMenus = 0;

    // group g: representative, people, accepted order (menu indexes, null if none), paid?
    private final String[] representatives = new String[MAX_GROUPS];
    private final int[] people = new int[MAX_GROUPS];
    private final int[][] orders = new int[MAX_GROUPS][];
    private final boolean[] paid = new boolean[MAX_GROUPS];
    private int nGroups = 0;

    public Restaurant(String name, int tables) {
        this.name = name;
        this.freeTables = tables;
    }

    // ----- R1 menus and reservations

    public void addMenu(String menu, double price) throws RestaurantException {
        if (indexOf(menuNames, nMenus, menu) >= 0) {
            throw new RestaurantException("duplicate menu " + menu);
        }
        if (nMenus == MAX_MENUS) {
            throw new IllegalStateException("full");
        }
        menuNames[nMenus] = menu;
        menuPrices[nMenus] = price;
        nMenus++;
    }

    public int reserve(String representative, int people) throws RestaurantException {
        if (indexOf(representatives, nGroups, representative) >= 0) {
            throw new RestaurantException("duplicate reservation " + representative);
        }
        if (people < 1) {
            throw new RestaurantException("invalid number of people " + people);
        }
        int tables = (people + SEATS_PER_TABLE - 1) / SEATS_PER_TABLE;   // ceiling with ints
        if (tables > freeTables) {
            refused += people;                 // the group is not stored: it can try again
            return 0;
        }
        if (nGroups == MAX_GROUPS) {
            throw new IllegalStateException("full");
        }
        representatives[nGroups] = representative;
        this.people[nGroups] = people;
```

```
        nGroups++;
        freeTables -= tables;
        return tables;
    }

    public int getRefused() {
        return refused;
    }

    public int getUnusedTables() {
        return freeTables;
    }

    // ----- R2 orders

    public boolean order(String representative, String... menus) throws RestaurantException {
        int g = group(representative);
        int[] order = new int[menus.length];
        for (int i = 0; i < menus.length; i++) {
            order[i] = indexOf(menuNames, nMenus, menus[i]);
            if (order[i] < 0) {
                throw new RestaurantException("unknown menu " + menus[i]);
            }
        }
        if (menus.length < people[g]) {
            return false;                       // refused: the previous order stays
        }
        orders[g] = order;
        paid[g] = false;                        // a new order is a new bill
        return true;
    }

    // ----- R3 bills and payments

    public String bill(String representative) throws RestaurantException {
        int g = groupWithOrder(representative);
        // "\n" and not %n: %n is "\r\n" on Windows, and the text asks for "\n"
        String bill = String.format(Locale.US, "%s - table for %s (%d people)\n",
                name, representative, people[g]);
        // counts[m] = how many times menu m is in the order; reset to 0 once its line is printed
        int[] counts = new int[nMenus];
        for (int m : orders[g]) {
            counts[m]++;
        }
        for (int m : orders[g]) {               // the order of first appearance
            if (counts[m] > 0) {
                bill += String.format(Locale.US, "%-20s%2d x %6.2f%8.2f\n",
                        menuNames[m], counts[m], menuPrices[m], counts[m] * menuPrices[m]);
                counts[m] = 0;
            }
        }
        if (people[g] >= 5) {
            // "Service 10%" is an argument, not part of the format: there % would be special
            bill += String.format(Locale.US, "%-31s%8.2f\n", "Service 10%", service(g));
        }
        bill += String.format(Locale.US, "%-31s%8.2f\n", "TOTAL", total(g));
        return bill;
    }

    public double pay(String representative) throws RestaurantException {
        int g = groupWithOrder(representative);
        if (paid[g]) {
            throw new RestaurantException("nothing to pay for " + representative);
```

```
        }
        double total = total(g);
        paid[g] = true;
        income = round(income + total);         // no 80.42999999999999 after many sums
        return total;
    }

    public double getIncome() {
        return income;
    }

    public String getUnpaid() {
        String[] unpaid = new String[nGroups];
        int n = 0;
        for (int g = 0; g < nGroups; g++) {
            if (orders[g] != null && !paid[g]) {
                unpaid[n++] = representatives[g];
            }
        }
        String[] result = Arrays.copyOf(unpaid, n);
        Arrays.sort(result);
        return String.join(", ", result);
    }

    // ----- helpers: bill() and pay() use the same computation, so they always agree

    private double subtotal(int g) {
        double subtotal = 0;
        for (int m : orders[g]) {
            subtotal += menuPrices[m];
        }
        return subtotal;
    }

    private double service(int g) {
        return people[g] >= 5 ? round(subtotal(g) * 0.10) : 0;
    }

    private double total(int g) {
        return round(subtotal(g) + service(g));
    }

    private static double round(double x) {
        return Math.round(x * 100) / 100.0;     // to the cent
    }

    private int group(String representative) throws RestaurantException {
        int g = indexOf(representatives, nGroups, representative);
        if (g < 0) {
            throw new RestaurantException("unknown representative " + representative);
        }
        return g;
    }

    private int groupWithOrder(String representative) throws RestaurantException {
        int g = group(representative);
        if (orders[g] == null) {
            throw new RestaurantException("no order for " + representative);
        }
        return g;
    }

    private static int indexOf(String[] array, int n, String value) {
```

```
        for (int i = 0; i < n; i++) {
            if (array[i].equals(value)) {
                return i;
            }
        }
        return -1;
    }
}
```

File `Main.java`:

```
import java.util.Locale;

public class Main {
    public static void main(String[] args) throws RestaurantException {
        Restaurant r = new Restaurant("Trattoria Da Gino", 3);
        r.addMenu("Pizza margherita", 7.5);
        r.addMenu("Tiramisu", 4.0);
        r.addMenu("Water", 1.0);
        IO.println(r.reserve("Rossi", 5) + " " + r.reserve("Bianchi", 6) + " "
                + r.reserve("Verdi", 2) + " refused=" + r.getRefused()
                + " unused=" + r.getUnusedTables());
        IO.println(r.order("Rossi", "Pizza margherita", "Tiramisu") + " "
                + r.order("Rossi", "Pizza margherita", "Tiramisu", "Pizza margherita", "Tiramisu",
                        "Pizza margherita"));
        r.order("Verdi", "Water", "Water");
        IO.print(r.bill("Rossi"));
        IO.print(r.bill("Verdi"));
        IO.println("unpaid: " + r.getUnpaid());
        IO.println(r.pay("Rossi") + " " + r.pay("Verdi")
                + String.format(Locale.US, " income=%.2f", r.getIncome())
                + " unpaid: [" + r.getUnpaid() + "]");
    }
}
```

The number of tables is $\lceil \text{people} / 4 \rceil$ = `(people + 3) / 4` in integer arithmetic; a refused group only adds to the refused count and is not stored, so it can try again. The bill lines use `"%-20s%2d x %6.2f%8.2f\n"` with `Locale.US`, and "\n" rather than `%n`, which is "\r\n" on Windows; `"Service 10%"` is passed as an argument to `"%-31s%8.2f\n"` because inside the format string `%` is special. To list each menu once, in order of first appearance, the bill counts the occurrences in an array and resets a count to 0 once its line is printed. `bill` and `pay` both call the same `total(g)` (subtotal, plus the service rounded with `Math.round(x * 100) / 100.0`, then rounded again), so the receipt and the payment always agree, and `getIncome` rounds after every payment so it never shows `80.42999999999999`. The text leaves open some messages (`duplicate menu`, `duplicate reservation`, `unknown representative`, `unknown menu`, `no order for`, `nothing to pay for`) and what happens with a new order after paying: here it becomes a new unpaid bill.

3.3 Supplier bids: StringBuilder, speed and the string pool

Package `ex3_3`.

File `BidException.java`:

```
public class BidException extends Exception {
    public BidException(String message) {
        super(message);
    }
}
```

File `Bids.java`:

```
// Exercise 3.3 - StringBuilder, equals vs ==, a sorted array
public class Bids {
    private static final int MAX_BIDS = 100;

    // always sorted by price, then by supplier
    private final String[] suppliers = new String[MAX_BIDS];
    private final int[] prices = new int[MAX_BIDS];
    private int nBids = 0;

    // ----- R1 bids

    public void addBid(String supplier, int price) throws BidException {
        if (price <= 0) {
            throw new BidException("invalid price " + price);
        }
        int old = indexOf(supplier);
        if (old < 0 && nBids == MAX_BIDS) {
            throw new IllegalStateException("full");      // checked before changing anything
        }
        if (old >= 0) {
            remove(old);                                   // the new bid replaces the old one
        }
        // shift right the bids that must come after the new one, then write it in the hole
        int i = nBids;
        while (i > 0 && comesAfter(i - 1, supplier, price)) {
            suppliers[i] = suppliers[i - 1];
            prices[i] = prices[i - 1];
            i--;
        }
        suppliers[i] = supplier;
        prices[i] = price;
        nBids++;
    }

    public boolean hasBid(String supplier) {
        return indexOf(supplier) >= 0;
    }

    // ----- R2 text

    public String getBids() {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < nBids; i++) {
            if (i > 0) {
                sb.append(',');                            // only between two bids
            }
            sb.append(suppliers[i]).append(':').append(prices[i]);
        }
        return sb.toString();
    }

    public String formatPrice(int cents) {
        // long: Math.abs(Integer.MIN_VALUE) would still be negative as an int
```

```
        StringBuilder sb = new StringBuilder(Long.toString(Math.abs((long) cents)));
        while (sb.length() < 3) {
            sb.insert(0, '0');                              // 5 -> 005, so that it becomes 0.05
        }
        sb.insert(sb.length() - 2, '.');
        // the integer part ends 3 characters from the end; a comma every 3 digits, right to left
        for (int i = sb.length() - 6; i > 0; i -= 3) {
            sb.insert(i, ',');
        }
        if (cents < 0) {
            sb.insert(0, '-');
        }
        return sb.toString();
    }

    // ----- R3 speed

    public String slowReport(int n) {
        String line = getBids() + "\n";
        String report = "";
        for (int i = 0; i < n; i++) {
            report += line;        // a new String each time: copies everything built so far
        }
        return report;
    }

    public String fastReport(int n) {
        String line = getBids() + "\n";
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) {
            sb.append(line);       // appends to the same buffer
        }
        return sb.toString();
    }

    // ----- helpers

    private int indexOf(String supplier) {
        for (int i = 0; i < nBids; i++) {
            // == would miss new String("s2") and "s" + "3"
            if (suppliers[i].equals(supplier)) {
                return i;
            }
        }
        return -1;
    }

    private void remove(int index) {
        for (int i = index; i < nBids - 1; i++) {
            suppliers[i] = suppliers[i + 1];
            prices[i] = prices[i + 1];
        }
        nBids--;
        suppliers[nBids] = null;
    }

    // does the bid at position i come after (supplier, price) in the order?
    private boolean comesAfter(int i, String supplier, int price) {
        if (prices[i] != price) {
            return prices[i] > price;
        }
        return suppliers[i].compareTo(supplier) > 0;
    }
```

```
}
```

File `Main.java`:

```
public class Main {
    public static void main(String[] args) throws BidException {
        Bids b = new Bids();
        b.addBid("s1", 120);
        b.addBid("s3", 120);
        b.addBid("s2", 100);
        b.addBid("s4", 90);
        b.addBid("s4", 150);                          // replaces 90
        IO.println(b.getBids());
        String half = "s";
        String[] names = {
            "s1", new String("s2"), half + "3", String.valueOf(new char[]{'s', '4'}), "s5"
        };
        String found = "";
        for (String n : names) {
            found += b.hasBid(n) ? "Y" : "n";
        }
        IO.println(found);
        IO.println(b.formatPrice(1234567) + " " + b.formatPrice(5) + " " + b.formatPrice(100000));
        try {
            b.addBid("s9", 0);
        } catch (BidException e) {
            IO.println("error: " + e.getMessage());
        }
        long t0 = System.currentTimeMillis();
        String slow = b.slowReport(10000);
        long t1 = System.currentTimeMillis();
        String fast = b.fastReport(10000);
        long t2 = System.currentTimeMillis();
        IO.println("same: " + slow.equals(fast) + ", length " + fast.length());
        IO.println("+= " + (t1 - t0) + " ms, StringBuilder " + (t2 - t1) + " ms");
    }
}
```

`hasBid` compares with `equals`: `new String("s2")`, `half + "3"` (computed at run time) and `String.valueOf(char[])` are new objects that are not in the string pool, so `==` would find only the literal `"s1"`. The array stays sorted by inserting each bid in its place, shifting to the right every bid with a higher price or the same price and a greater name; a replacement first removes the old bid and then inserts the new one, and capacity is checked before anything is removed. `formatPrice` pads the digits to at least three, inserts the point two places from the end, then inserts a comma every three digits going right to left, so the positions still to be processed never move; it also handles negative amounts by working on `Math.abs((long) cents)`, since `Math.abs(Integer.MIN_VALUE)` is still negative as an `int`. The two reports compute the line once; `+=` copies the whole string built so far at every step, which takes quadratic time (about a second for 10,000 lines here), while `StringBuilder.append` writes into one buffer and takes a few milliseconds.

3.4 Importing contacts: encodings

Package `ex3_4`.

File `ImportException.java`:

```
public class ImportException extends Exception {
    public ImportException(String message) {
        super(message);
    }
}
```

File `AddressBook.java`:

```
import java.nio.charset.Charset;
import java.nio.charset.CharsetEncoder;
import java.nio.charset.StandardCharsets;
import java.util.Arrays;

// Exercise 3.4 - bytes, charsets, code points
public class AddressBook {
    private static final int MAX_CONTACTS = 100;
    private static final int MAX_NAME_CODE_POINTS = 20;

    private final String[] names = new String[MAX_CONTACTS];
    private final String[] emails = new String[MAX_CONTACTS];
    private int nContacts = 0;

    // ----- R1 import

    public int importData(byte[] data, String charsetName) throws ImportException {
        String text = new String(data, Charset.forName(charsetName));   // bytes -> chars
        String[] lines = text.split("\n");
        // check every line first and store only at the end: a bad line adds nothing
        String[] newNames = new String[lines.length];
        String[] newEmails = new String[lines.length];
        int added = 0;
        for (int i = 0; i < lines.length; i++) {
            String line = lines[i];
            if (line.isEmpty()) {
                continue;                       // skipped, but it still counts as a line
            }
            int semicolon = line.indexOf(';');
            if (semicolon < 0 || semicolon != line.lastIndexOf(';')) {
                throw new ImportException("bad line " + (i + 1) + ": " + line);
            }
            String name = repair(line.substring(0, semicolon));
            // length() counts chars: an emoji is 2 chars but 1 code point
            if (name.codePointCount(0, name.length()) > MAX_NAME_CODE_POINTS) {
                throw new ImportException("name too long: " + name);
            }
            newNames[added] = name;
            newEmails[added] = line.substring(semicolon + 1);
            added++;
        }
        if (nContacts + added > MAX_CONTACTS) {
            throw new IllegalStateException("full");
        }
        for (int i = 0; i < added; i++) {
            names[nContacts] = newNames[i];
            emails[nContacts] = newEmails[i];
            nContacts++;
        }
        return added;
    }

    // ----- R2 repair
```

```
    // "MÃ¼ller" is "Müller" written in UTF-8 and read as ISO-8859-1: undo the wrong decoding
    public static String repair(String s) {
        // a char outside ISO-8859-1 cannot come from that mistake
        // (and getBytes would turn it into '?')
        if (s.indexOf('Ã') < 0 || !StandardCharsets.ISO_8859_1.newEncoder().canEncode(s)) {
            return s;
        }
        String fixed = new String(s.getBytes(StandardCharsets.ISO_8859_1), StandardCharsets.UTF_8);
        return fixed.indexOf('') < 0 ? fixed : s;  // U+FFFD: the bytes were not valid UTF-8
    }

    // ----- R3 export

    public byte[] export(String charsetName) throws ImportException {
        Charset charset = Charset.forName(charsetName);
        CharsetEncoder encoder = charset.newEncoder();
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < nContacts; i++) {
            // getBytes would silently write '?' for a character the charset does not have
            if (!encoder.canEncode(names[i])) {
                throw new ImportException("cannot encode " + names[i] + " in " + charsetName);
            }
            if (!encoder.canEncode(emails[i])) {
                throw new ImportException("cannot encode " + emails[i] + " in " + charsetName);
            }
            sb.append(names[i]).append(';').append(emails[i]).append('\n');
        }
        return sb.toString().getBytes(charset);
    }

    public static String hex(byte[] bytes) {
        StringBuilder sb = new StringBuilder();
        for (byte b : bytes) {
            if (sb.length() > 0) {
                sb.append(' ');
            }
            // & 0xFF: a byte is signed, (byte) 0xC3 is -61
            sb.append(String.format("%02X", b & 0xFF));
        }
        return sb.toString();
    }

    public String names() {
        return String.join(", ", Arrays.copyOf(names, nContacts));
    }
}
```

File `Main.java`:

```
import java.nio.charset.StandardCharsets;

public class Main {
    public static void main(String[] args) throws ImportException {
        AddressBook book = new AddressBook();
        byte[] utf8 = "José;jose@mail.es\nZoë 😀;zoe@mail.com\n\nPerché;why@mail.it\n"
                .getBytes(StandardCharsets.UTF_8);
        IO.println(book.importData(utf8, "UTF-8") + " " + book.names());
        String broken = new String("Müller;mueller@mail.de".getBytes(StandardCharsets.UTF_8),
                StandardCharsets.ISO_8859_1);
        IO.println(broken + " -> " + AddressBook.repair(broken.split(";")[0]));
        book.importData(broken.getBytes(StandardCharsets.ISO_8859_1), "ISO-8859-1");
        IO.println(book.names());
        IO.println(AddressBook.hex("é€".getBytes(StandardCharsets.UTF_8)) + " | "
                + book.export("UTF-8").length);
        try {
            book.export("ISO-8859-1");
        } catch (ImportException e) {
            IO.println("error: " + e.getMessage());
        }
        try {
            book.importData("a;b;c\n".getBytes(StandardCharsets.UTF_8), "UTF-8");
        } catch (ImportException e) {
            IO.println("error: " + e.getMessage());
        }
    }
}
```

`importData` decodes once with `new String(data, Charset.forName(name))`, splits on "\n" and numbers the lines from 1 counting the empty ones too. A line is valid when `indexOf(';')` is not -1 and equals `lastIndexOf(';')`; like 3.1, the import is all or nothing: every line is checked before any contact is stored. The name limit uses `codePointCount`, because `length()` counts UTF-16 chars and an emoji is two of them; it is checked after `repair`, on the real name. `repair` undoes "UTF-8 bytes read as ISO-8859-1" by re-encoding in ISO-8859-1 and decoding as UTF-8, and keeps the original if the result has `'�'`. It also leaves alone a string with a character outside ISO-8859-1 (not in the text), because `getBytes` would silently turn that character into `'?'`; for the same reason `export` checks every name (and email) with `charset.newEncoder().canEncode(...)` before calling `getBytes`, and `hex` uses `b & 0xFF`, because `byte` is signed.

Part 4 – JUnit 5

4.1 Exam-style tests for a parking lot

Package `ex4_1`.

The tests (`test/ex4_1`):

File `TestR1_Parking.java`:

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

public class TestR1_Parking {
```

```
    private static final double DELTA = 1e-9;   // doubles are compared with a tolerance

    private Parking parking;

    @BeforeEach
    public void setUp() {
        parking = new Parking(3);   // a new object for every test: no test depends on another
    }

    @Test
    public void testFirstTickets() throws ParkingException {
        assertEquals(1, parking.enter("AB123CD", 0));
        assertEquals(2, parking.enter("ZZ999ZZ", 10));
    }

    @Test
    public void testFreeSpots() throws ParkingException {
        assertEquals(3, parking.freeSpots());
        parking.enter("AB123CD", 0);
        assertEquals(2, parking.freeSpots());
        parking.enter("ZZ999ZZ", 0);
        assertEquals(1, parking.freeSpots());
    }

    @Test
    public void testAlreadyInside() throws ParkingException {
        parking.enter("AB123CD", 0);
        assertThrows(ParkingException.class, () -> parking.enter("AB123CD", 10));
        // nothing changed: same free spots, same plates, and no ticket was used up
        assertEquals(2, parking.freeSpots());
        assertEquals("AB123CD", parking.plates());
        assertEquals(2, parking.enter("ZZ999ZZ", 20));
    }

    @Test
    public void testFull() throws ParkingException {
        parking.enter("AA111AA", 0);
        parking.enter("BB222BB", 0);
        parking.enter("CC333CC", 0);
        assertEquals(0, parking.freeSpots());
        assertEquals(-1, parking.enter("DD444DD", 5));
    }

    @Test
    public void testFee() throws ParkingException {
        parking.enter("AA111AA", 0);
        assertEquals(1.5, parking.exit("AA111AA", 1), DELTA);
        parking.enter("AA111AA", 100);
        assertEquals(1.5, parking.exit("AA111AA", 160), DELTA);
        parking.enter("AA111AA", 0);
        assertEquals(3.0, parking.exit("AA111AA", 61), DELTA);
        parking.enter("AA111AA", 0);
        assertEquals(4.5, parking.exit("AA111AA", 180), DELTA);
    }

    @Test
    public void testExitNotInside() throws ParkingException {
        ParkingException e = assertThrows(ParkingException.class,
                () -> parking.exit("XY000XY", 30));
        assertEquals("not inside: XY000XY", e.getMessage());
    }
```

```
    @Test
    public void testPlates() throws ParkingException {
        assertEquals("", parking.plates());
        parking.enter("ZZ999ZZ", 0);
        parking.enter("AB123CD", 5);
        assertEquals("AB123CD, ZZ999ZZ", parking.plates());
    }

    @Test
    public void testTicketsPerParking() throws ParkingException {
        parking.enter("AB123CD", 0);
        parking.enter("ZZ999ZZ", 0);
        Parking other = new Parking(2);
        assertEquals(1, other.enter("AB123CD", 0));   // the counter is not static
    }
}
```

The `@BeforeEach` creates a new `Parking(3)` before every test, so the tests are independent and can run in any order; each test checks one point and declares `throws ParkingException` instead of catching it, so an unexpected exception turns the test red. Exceptions are tested with `assertThrows`, which also returns the exception: that is how the message `not inside: XY000XY` is compared, and how test 3 then checks that the failed `enter` changed nothing (free spots, plates, and the next ticket is still 2). Fees are doubles, so they are compared with `assertEquals(expected, actual, delta)`, and the boundaries are tested on both sides: 1 and 60 minutes cost 1.50, 61 minutes 3.00. Test 8 is the one that catches a `static` ticket counter, a classic exam bug; each of these mistakes (static counter, a floor division in the fee, a missing sort, a ticket used by a failed entry, a different message, 0 instead of -1 when full) makes at least one test fail.

4.2 Testing the food delivery of 2.8, then a suite

Package `ex4_2`.

The tests (`test/ex4_2`):

File `TestR1_Menu.java`:

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import ex2_8.*;

public class TestR1_Menu {
    private Delivery d;

    @BeforeEach
    public void setUp() throws DeliveryException {
        // a new object for every test: no test sees another's data
        d = new Delivery();
        d.addItem("Chinese noodles", Category.MAIN_COURSE, 8.5, 20);
        d.addItem("Spring rolls", Category.STARTER, 4.0, 10);
        d.addItem("Green tea", Category.DRINK, 2.5, 2);
        d.addItem("Mochi", Category.DESSERT, 3.0, 5);
    }

    @Test
    public void testFindIgnoresCase() throws DeliveryException {
```

```
        assertEquals("Chinese noodles", d.findItem("NOODLE").description());
        assertEquals("Green tea", d.findItem("gReEn").description());
    }

    @Test
    public void testFindNoMatch() {
        DeliveryException e = assertThrows(DeliveryException.class, () -> d.findItem("pizza"));
        assertEquals("no item matches pizza", e.getMessage());
    }

    @Test
    public void testFindTwoMatches() {
        // "Chinese noodles" and "Mochi" both contain "ch"
        DeliveryException e = assertThrows(DeliveryException.class, () -> d.findItem("CH"));
        assertEquals("2 items match CH", e.getMessage());
    }

    @Test
    public void testDuplicateItem() throws DeliveryException {
        assertThrows(DeliveryException.class,
                () -> d.addItem("Spring rolls", Category.STARTER, 9.0, 1));
        // the failed add changed nothing: the old price is still there
        assertEquals("[STARTER] Spring rolls : 4.00", d.findItem("rolls").toString());
    }
}
```

File `TestR2_Orders.java`:

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import ex2_8.*;

public class TestR2_Orders {
    private Delivery d;

    @BeforeEach
    public void setUp() throws DeliveryException {
        d = new Delivery();
        d.addItem("Chinese noodles", Category.MAIN_COURSE, 8.5, 20);
        d.addItem("Spring rolls", Category.STARTER, 4.0, 10);
        d.addItem("Green tea", Category.DRINK, 2.5, 2);
        d.addItem("Mochi", Category.DESSERT, 3.0, 5);
    }

    @Test
    public void testFirstIds() {
        assertEquals(1, d.newOrder());
        assertEquals(2, d.newOrder());
    }

    @Test
    public void testSameItemTwice() throws DeliveryException {
        int o = d.newOrder();
        assertEquals(2, d.addToOrder(o, "noodles", 2));
        assertEquals(5, d.addToOrder(o, "NOODLES", 3));
        assertEquals("Chinese noodles, 5", d.showOrder(o));   // one line, not two
    }
```

```
    @Test
    public void testShowOrder() throws DeliveryException {
        int o = d.newOrder();
        d.addToOrder(o, "noodles", 2);
        d.addToOrder(o, "rolls", 1);
        assertEquals("Chinese noodles, 2; Spring rolls, 1", d.showOrder(o));
    }

    @Test
    public void testTotal() throws DeliveryException {
        int o = d.newOrder();
        d.addToOrder(o, "noodles", 3);
        d.addToOrder(o, "rolls", 1);
        assertEquals(29.5, d.total(o), 0.001);                // doubles: always with a tolerance
    }

    @Test
    public void testUnknownOrder() {
        int unknown = d.newOrder() + 1;
        assertThrows(DeliveryException.class, () -> d.addToOrder(unknown, "tea", 1));
        assertThrows(DeliveryException.class, () -> d.total(unknown));
        assertThrows(DeliveryException.class, () -> d.showOrder(unknown));
    }
}
```

File `TestR3_Status.java`:

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertSame;
import static org.junit.jupiter.api.Assertions.assertThrows;

import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import ex2_8.*;

public class TestR3_Status {
    // static: JUnit creates a new TestR3_Status object for every test
    private static int tests;
    private Delivery d;

    @BeforeAll
    public static void start() {
        IO.println("R3: start");
        tests = 0;
    }

    @AfterAll
    public static void end() {
        IO.println("R3: " + tests + " tests");
    }

    @BeforeEach
    public void setUp() throws DeliveryException {
        d = new Delivery();
        d.addItem("Chinese noodles", Category.MAIN_COURSE, 8.5, 20);
        d.addItem("Spring rolls", Category.STARTER, 4.0, 10);
        d.addItem("Green tea", Category.DRINK, 2.5, 2);
        d.addItem("Mochi", Category.DESSERT, 3.0, 5);
```

```
    }

    @AfterEach
    public void countTest() {
        tests++;
    }

    @Test
    public void testNewOrderIsNew() throws DeliveryException {
        int o = d.newOrder();
        assertSame(Status.NEW, d.status(o));                  // enum constants are unique objects
    }

    @Test
    public void testMoves() throws DeliveryException {
        int o = d.newOrder();
        d.addToOrder(o, "noodles", 1);
        d.addToOrder(o, "tea", 2);
        assertEquals(40, d.confirm(o));
        assertEquals(35, d.start(o));
        assertEquals(15, d.deliver(o));
        assertEquals(0, d.complete(o));
        assertSame(Status.DELIVERED, d.status(o));
    }

    @Test
    public void testWrongState() throws DeliveryException {
        int o = d.newOrder();
        DeliveryException e = assertThrows(DeliveryException.class, () -> d.start(o));
        assertEquals("order 1 is NEW, expected CONFIRMED", e.getMessage());
        assertSame(Status.NEW, d.status(o));                  // the failed move changed nothing
    }

    @Test
    public void testConfirmAfterComplete() throws DeliveryException {
        int o = d.newOrder();
        d.addToOrder(o, "noodles", 1);
        d.confirm(o);
        d.start(o);
        d.deliver(o);
        d.complete(o);
        assertThrows(DeliveryException.class, () -> d.confirm(o));
    }
}
```

File `AllTests.java`:

```
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;

// runs the three classes together, as the exam's test suite does
@Suite
@SelectClasses({ TestR1_Menu.class, TestR2_Orders.class, TestR3_Status.class })
public class AllTests {
}
```

Each class builds a new `Delivery` in its `@BeforeEach`, so every test starts from the four items and no orders, and the exceptions are checked with `assertThrows`, whose return value gives the message (`no item matches pizza`, `2 items match CH`, `order 1 is NEW, expected CONFIRMED`). `total` is compared with a tolerance, the enum states with `assertSame`, and the duplicate-item test also checks that the old item is unchanged. In `TestR3_Status` the counter must be `static`, like the `@BeforeAll`/`@AfterAll` methods, because JUnit creates a new test object for every test, so an instance field would always be 1. With a `static` order counter in `Delivery`, `AllTests` gives 11 green and 2 red: `testFirstIds` (expected 1 but was 4) and `testWrongState` (got `order 8 is NEW, ...`), and each one fails even when its class runs alone, because JUnit's method order is not the source order and another test creates orders first.

4.3 Bug reproduction and TDD

Package `ex4_3`.

File `Ranking.java`:

```
public class Ranking {

    /** Decreasing score, ties in alphabetical order (Ann:9,Bob:7,Zoe:7); "" with no players. */
    public static String format(String[] names, int[] scores) {
        String[] n = names.clone();
        int[] s = scores.clone();
        // selection sort: position i gets the best of i..end
        for (int i = 0; i < n.length; i++) {
            for (int j = i + 1; j < n.length; j++) {
                // fix 1: higher score first (was s[j] < s[i]); fix 2: on a tie, the name decides
                if (s[j] > s[i] || (s[j] == s[i] && n[j].compareTo(n[i]) < 0)) {
                    String tn = n[i]; n[i] = n[j]; n[j] = tn;
                    int ts = s[i]; s[i] = s[j]; s[j] = ts;
                }
            }
        }
        // fix 3: start from "" (no n[0] with no players)
        // and put the comma before every player but the first
        String r = "";
        for (int i = 0; i < n.length; i++) {
            if (i > 0) {
                r += ",";
            }
            r += n[i] + ":" + s[i];
        }
        return r;
    }
}
```

File `TimeUtils.java`:

```java
import java.util.Locale;

public class TimeUtils {

    public static final int MINUTES_PER_DAY = 24 * 60;

    /** "08:15" -> 495. Exactly two digits, ':', two digits; hours 0-23, minutes 0-59. */
    public static int toMinutes(String hhmm) {
        // check the shape first: then parseInt cannot fail,
        // and null gets the same exception as bad text
        if (hhmm == null || hhmm.length() != 5 || hhmm.charAt(2) != ':'
                || !isDigit(hhmm.charAt(0)) || !isDigit(hhmm.charAt(1))
                || !isDigit(hhmm.charAt(3)) || !isDigit(hhmm.charAt(4))) {
            throw new IllegalArgumentException("invalid time: " + hhmm);
        }
        int hours = Integer.parseInt(hhmm.substring(0, 2));
        int minutes = Integer.parseInt(hhmm.substring(3));
        if (hours > 23 || minutes > 59) {
            throw new IllegalArgumentException("invalid time: " + hhmm);
        }
        return hours * 60 + minutes;
    }

    // only '0'..'9': Character.isDigit would also accept the digits of other scripts
    private static boolean isDigit(char c) {
        return c >= '0' && c <= '9';
    }

    /** 495 -> "08:15". */
    public static String toText(int minutes) {
        if (minutes < 0 || minutes >= MINUTES_PER_DAY) {
            throw new IllegalArgumentException("minutes must be 0-1439: " + minutes);
        }
        return String.format(Locale.US, "%02d:%02d", minutes / 60, minutes % 60);
    }

    /** The minutes from 'from' to 'to', past midnight if needed: "23:30" -> "00:15" is 45. */
    public static int duration(String from, String to) {
        // + a whole day before the % keeps the result non-negative
        return (toMinutes(to) - toMinutes(from) + MINUTES_PER_DAY) % MINUTES_PER_DAY;
    }

    /** "1h30m" -> 90. Not written yet: its test is @Disabled. */
    public static int parseDuration(String s) {
        throw new UnsupportedOperationException("parseDuration not implemented yet");
    }
}
```

The tests (`test/ex4_3`):

File `RankingTest.java`:

```
import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.Test;

public class RankingTest {

    @Test
    public void testBugReport() {   // the exact case of the bug report
        assertEquals("Ann:9,Bob:7,Zoe:7",
                Ranking.format(new String[]{"Zoe", "Ann", "Bob"}, new int[]{7, 9, 7}));
    }

    @Test
    public void testTieAlphabetical() {
        assertEquals("Bob:7,Zoe:7", Ranking.format(new String[]{"Zoe", "Bob"}, new int[]{7, 7}));
    }

    @Test
    public void testNoPlayers() {
        assertEquals("", Ranking.format(new String[0], new int[0]));
    }

    @Test
    public void testOnePlayer() {
        assertEquals("Ann:9", Ranking.format(new String[]{"Ann"}, new int[]{9}));
    }
}
```

File `TimeUtilsTest.java`:

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.junit.jupiter.api.Assumptions.assumeTrue;

import org.junit.jupiter.api.Disabled;
import org.junit.jupiter.api.Test;

public class TimeUtilsTest {

    @Test
    public void testToMinutes() {
        assertEquals(0, TimeUtils.toMinutes("00:00"));
        assertEquals(495, TimeUtils.toMinutes("08:15"));
        assertEquals(1439, TimeUtils.toMinutes("23:59"));
    }

    @Test
    public void testToMinutesInvalid() {
        assertThrows(IllegalArgumentException.class, () -> TimeUtils.toMinutes("8:15"));
        assertThrows(IllegalArgumentException.class, () -> TimeUtils.toMinutes("24:00"));
        assertThrows(IllegalArgumentException.class, () -> TimeUtils.toMinutes("12:60"));
        assertThrows(IllegalArgumentException.class, () -> TimeUtils.toMinutes("ab:cd"));
        assertThrows(IllegalArgumentException.class, () -> TimeUtils.toMinutes("1215"));
        assertThrows(IllegalArgumentException.class, () -> TimeUtils.toMinutes(null));
    }

    @Test
    public void testToText() {
        assertEquals("00:00", TimeUtils.toText(0));
        assertEquals("08:15", TimeUtils.toText(495));
        assertEquals("23:59", TimeUtils.toText(1439));
```

```
    }

    @Test
    public void testToTextOutOfRange() {
        assertThrows(IllegalArgumentException.class, () -> TimeUtils.toText(-1));
        assertThrows(IllegalArgumentException.class, () -> TimeUtils.toText(1440));
    }

    @Test
    public void testDuration() {
        assertEquals(45, TimeUtils.duration("08:15", "09:00"));
        assertEquals(45, TimeUtils.duration("23:30", "00:15"));   // past midnight
    }

    @Test
    @Disabled("parseDuration not implemented yet")
    public void testParseDuration() {
        assertEquals(90, TimeUtils.parseDuration("1h30m"));
    }

    @Test
    public void exhaustiveCheck() {
        // aborted (not failed) unless SLOW_TESTS is set
        assumeTrue(System.getenv("SLOW_TESTS") != null, "set SLOW_TESTS to run it");
        for (int m = 0; m < 1440; m++) {
            assertEquals(m, TimeUtils.toMinutes(TimeUtils.toText(m)));
        }
    }
}
```

The buggy `format` has three mistakes: the sort compares `s[j] < s[i]` (increasing score, and no rule for ties), the result starts from `n[0]` (an `ArrayIndexOutOfBoundsException` with no players) and the comma goes after every player but the first; the fix changes the condition to "higher score, or same score and smaller name" and builds the string from `""`, adding the comma before every player except the first. On the buggy code the two wrong strings are failures (an `AssertionFailedError`), the crash is an error (an unexpected exception: the JUnit 5 console counts both as "failed", Eclipse shows them apart) and the single player passes by luck. In `toMinutes` the `null` check comes first, then the shape (length 5, `:` at position 2, a digit from `'0'` to `'9'` everywhere else; `Character.isDigit` would also accept the digits of other scripts), and only then the ranges: splitting at `:` and calling `Integer.parseInt` would accept `"8:15"` and `"+8:15"`, and `null` would give a `NullPointerException` instead of the required `IllegalArgumentException`. `duration` adds 1440 before the `%`, because in Java `-1395 % 1440` is negative; `toText` uses `Locale.US` so the digits are always ASCII. `@Disabled` gives a skipped test, while `assumeTrue(System.getenv("SLOW_TESTS") != null)` gives an aborted one, never a failure.

Part 5 – Inheritance, interfaces, lambdas

5.1 Bank accounts: inheritance

Package `ex5_1`.

File `BankException.java`:

```java
public class BankException extends Exception {
    public BankException(String message) {
        super(message);
    }
}
```

File `Account.java`:

```java
import java.util.Locale;

public class Account {

    private final int number;
    private final String owner;
    // private even for the subclasses: they go through getBalance() and charge()
    private double balance;

    public Account(int number, String owner, double initial) {
        this.number = number;
        this.owner = owner;
        this.balance = initial;
    }

    public int getNumber() { return number; }

    public String getOwner() { return owner; }

    public double getBalance() { return balance; }

    public void deposit(double amount) {
        balance += amount;
    }

    // the rule is fixed here, the condition is a hook that the subclasses redefine
    public void withdraw(double amount) throws BankException {
        if (!canWithdraw(amount)) {
            throw new BankException("insufficient funds on " + number);
        }
        balance -= amount;
    }

    protected boolean canWithdraw(double amount) {
        return balance - amount >= 0;
    }

    protected void charge(double amount) {
        balance -= amount;
    }

    public double monthlyFee() {
        return 1.0;
    }

    public String describe() {
        return String.format(Locale.US, "%d,%s,%.2f", number, owner, balance);
    }

    @Override
    public String toString() {
        return describe();   // dynamic binding: a subclass's describe() is used
    }
```

```
}
```

File `CheckingAccount.java`:

```
import java.util.Locale;

public class CheckingAccount extends Account {

    private final double overdraft;

    public CheckingAccount(int number, String owner, double initial, double overdraft) {
        super(number, owner, initial);   // must be the first statement
        this.overdraft = overdraft;
    }

    public double getOverdraft() { return overdraft; }

    @Override
    protected boolean canWithdraw(double amount) {
        return getBalance() - amount >= -overdraft;
    }

    @Override
    public double monthlyFee() {
        return 2.0;
    }

    @Override
    public String describe() {
        return super.describe() + String.format(Locale.US, ",overdraft %.2f", overdraft);
    }
}
```

File `PremiumAccount.java`:

```
public class PremiumAccount extends CheckingAccount {

    public PremiumAccount(int number, String owner, double initial, double overdraft) {
        super(number, owner, initial, overdraft);
    }

    @Override
    public double monthlyFee() {
        return 0;
    }

    @Override
    public String describe() {
        // super is CheckingAccount: the overdraft is already there
        return super.describe() + ",premium";
    }
}
```

File SavingsAccount.java:

```
public class SavingsAccount extends Account {

    private final double rate;   // percent per year

    public SavingsAccount(int number, String owner, double initial, double rate) {
        super(number, owner, initial);
        this.rate = rate;
    }

    public double getRate() { return rate; }

    public void addInterest() {
        deposit(getBalance() * rate / 100);
    }

    @Override
    public double monthlyFee() {
        return 0;
    }

    @Override
    public String describe() {
        // double concatenation: 2.0, never locale-dependent
        return super.describe() + ",rate " + rate + "%";
    }
}
```

File Bank.java:

```
public class Bank {

    private static final int MAX_ACCOUNTS = 100;

    private final Account[] accounts = new Account[MAX_ACCOUNTS];
    private int count = 0;

    public int open(String owner, double initial) {
        return add(new Account(count + 1, owner, initial));
    }

    public int openChecking(String owner, double initial, double overdraft) {
        return add(new CheckingAccount(count + 1, owner, initial, overdraft));
    }

    public int openSavings(String owner, double initial, double rate) {
        return add(new SavingsAccount(count + 1, owner, initial, rate));
    }

    public int openPremium(String owner, double initial, double overdraft) {
        return add(new PremiumAccount(count + 1, owner, initial, overdraft));
    }

    // one array of the superclass holds every kind of account
    private int add(Account a) {
        if (count == accounts.length) {
            throw new IllegalStateException("full");
        }
        accounts[count++] = a;
        return a.getNumber();
    }
```

```
    public Account getAccount(int number) throws BankException {
        if (number < 1 || number > count) {
            throw new BankException("unknown account " + number);
        }
        return accounts[number - 1];   // numbers are 1, 2, ...: account n is in cell n - 1
    }

    public void chargeFees() {
        for (int i = 0; i < count; i++) {
            accounts[i].charge(accounts[i].monthlyFee());   // each account's own fee
        }
    }

    public int countChecking() {
        int n = 0;
        for (int i = 0; i < count; i++) {
            if (accounts[i] instanceof CheckingAccount) {   // true for a PremiumAccount too
                n++;
            }
        }
        return n;
    }

    public SavingsAccount bestSavingsOf(String owner) {
        SavingsAccount best = null;
        for (int i = 0; i < count; i++) {
            if (accounts[i] instanceof SavingsAccount s && s.getOwner().equals(owner)
                    && (best == null || s.getBalance() > best.getBalance())) {
                best = s;
            }
        }
        return best;
    }

    public double totalDeposits() {
        double total = 0;
        for (int i = 0; i < count; i++) {
            total += accounts[i].getBalance();
        }
        return total;
    }
}
```

File `Main.java`:

```
import java.util.Locale;

public class Main {
    public static void main(String[] args) throws BankException {
        Bank bank = new Bank();
        int a = bank.open("Paul", 100);
        int c = bank.openChecking("Anna", 50, 500);
        int s = bank.openSavings("Paul", 1000, 2);
        int p = bank.openPremium("Luca", 0, 1000);
        bank.getAccount(c).withdraw(300);
        try {
            bank.getAccount(a).withdraw(300);
        } catch (BankException e) {
            IO.println("error: " + e.getMessage());
        }
        ((SavingsAccount) bank.getAccount(s)).addInterest();
        bank.chargeFees();
        for (int n = 1; n <= 4; n++) {
            IO.println(bank.getAccount(n));
        }
        IO.println(bank.countChecking() + " " + bank.bestSavingsOf("Paul") + " | "
                + bank.bestSavingsOf("Anna"));
        IO.println(String.format(Locale.US, "total %.2f", bank.totalDeposits()));
    }
}
```

`withdraw` is written once in `Account` and asks the `protected` hook `canWithdraw` whether it may proceed, so `CheckingAccount` changes only the condition (balance down to −overdraft) and `PremiumAccount` inherits it; the balance stays `private`, the subclasses read it with `getBalance()`, and the fees use `charge`, which skips the check, so a fee is never refused (a plain account with 0.00 goes to −1.00). Each `describe()` extends `super.describe()`, so the premium account gets `,overdraft 1000.00,premium` without repeating any code, and `toString()` in `Account` calls `describe()`, which dynamic binding resolves to the subclass's version. The `Bank` keeps all kinds of accounts in one `Account[]`; `chargeFees` calls `monthlyFee()` without asking the type, `countChecking` uses `instanceof CheckingAccount` (true for a premium account too), and `bestSavingsOf` uses `instanceof SavingsAccount s` to test and cast in one step. The amounts use `%.2f` with `Locale.US`, while the rate is plain string concatenation of the double (`2.0`, which is locale-independent), the reading that matches `rate 2.0%`. The balances are `double`s compared as they are, with no tolerance, as the exams usually do: open with 0.30, withdraw 0.10, and the balance is 0.19999999999999998, so a further withdrawal of 0.20 is refused; counting cents in a `long`, as in 2.7, is the exact alternative.

5.2 Customers and orders: equals, hashCode, toString, clone

Package `ex5_2`.

File `ShopException.java`:

```
public class ShopException extends Exception {
    public ShopException(String message) {
        super(message);
    }
}
```

File `Customer.java`:

```java
import java.util.Locale;

// immutable: final class, final fields, no setters
public final class Customer {

    private final String name;
    private final String email;

    public Customer(String name, String email) {
        this.name = name;
        this.email = email;
    }

    public String getName() { return name; }

    public String getEmail() { return email; }

    // equals and hashCode are both computed from this key,
    // so equal customers always have equal hashes
    // (Locale.ROOT: with a Turkish default locale, "I".toLowerCase() is "ı", not "i")
    private String key() {
        return email.toLowerCase(Locale.ROOT);
    }

    @Override
    public boolean equals(Object o) {
        // the parameter is Object: equals(Customer) would be an overload,
        // which countDistinct(Object...) never calls;
        // instanceof is false for null and for other classes, e.g. a String
        return o instanceof Customer other && key().equals(other.key());
    }

    @Override
    public int hashCode() {
        return key().hashCode();
    }

    @Override
    public String toString() {
        return name + " <" + email + ">";
    }
}
```

File `Order.java`:

```java
public class Order implements Cloneable {

    private static final int MAX_LINES = 10;

    int id;                          // package-private and not final: Shop gives a copy a new id
    private final Customer customer;  // shared by the copies: a Customer never changes
    // not final: clone() replaces the arrays in the copy
    private String[] items = new String[MAX_LINES];
    private int[] qty = new int[MAX_LINES];
    private int lines = 0;

    public Order(int id, Customer customer) {
        this.id = id;
        this.customer = customer;
    }
```

```
    public int getId() { return id; }

    public Customer getCustomer() { return customer; }

    public void add(String item, int q) {
        for (int i = 0; i < lines; i++) {
            if (items[i].equals(item)) {
                qty[i] += q;
                return;
            }
        }
        if (lines == MAX_LINES) {
            throw new IllegalStateException("full");
        }
        items[lines] = item;
        qty[lines] = q;
        lines++;
    }

    @Override
    public Order clone() {
        try {
            // field by field: the copy still shares the arrays
            Order copy = (Order) super.clone();
            copy.items = items.clone();           // deep copy of the mutable parts
            copy.qty = qty.clone();
            return copy;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);          // cannot happen: Order implements Cloneable
        }
    }

    @Override
    public String toString() {
        String s = "#" + id + " " + customer.getEmail() + ":";
        for (int i = 0; i < lines; i++) {
            s += (i == 0 ? " " : ", ") + qty[i] + " x " + items[i];
        }
        return s;
    }
}
```

File `Shop.java`:

```
import java.util.Objects;

public class Shop {

    private static final int MAX_CUSTOMERS = 100;
    private static final int MAX_ORDERS = 100;

    private final Customer[] customers = new Customer[MAX_CUSTOMERS];
    private int nCustomers = 0;
    private final Order[] orders = new Order[MAX_ORDERS];
    private int nOrders = 0;

    public void addCustomer(String name, String email) throws ShopException {
        Customer c = new Customer(name, email);
        if (find(c) != null) {
            throw new ShopException("customer already registered: " + email);
        }
        if (nCustomers == customers.length) {
```

```
            throw new IllegalStateException("full");
        }
        customers[nCustomers++] = c;
    }

    public int newOrder(String email) throws ShopException {
        // equals looks only at the email,
        // so a "probe" customer with an empty name finds the real one
        Customer c = find(new Customer("", email));
        if (c == null) {
            throw new ShopException("unknown customer: " + email);
        }
        return addOrder(new Order(nOrders + 1, c));
    }

    public void addItem(int orderId, String item, int q) throws ShopException {
        getOrder(orderId).add(item, q);
    }

    public int repeat(int orderId) throws ShopException {
        Order copy = getOrder(orderId).clone();
        copy.id = nOrders + 1;
        return addOrder(copy);
    }

    public String show(int orderId) throws ShopException {
        return getOrder(orderId).toString();
    }

    public static int countDistinct(Object... items) {
        int n = 0;
        for (int i = 0; i < items.length; i++) {
            boolean seen = false;
            for (int j = 0; j < i && !seen; j++) {
                seen = Objects.equals(items[j], items[i]);   // null-safe equals
            }
            if (!seen) {
                n++;
            }
        }
        return n;
    }

    private Customer find(Customer probe) {
        for (int i = 0; i < nCustomers; i++) {
            if (customers[i].equals(probe)) {
                return customers[i];
            }
        }
        return null;
    }

    private int addOrder(Order o) {
        if (nOrders == orders.length) {
            throw new IllegalStateException("full");
        }
        orders[nOrders++] = o;
        return o.id;
    }

    private Order getOrder(int orderId) throws ShopException {
        if (orderId < 1 || orderId > nOrders) {
            throw new ShopException("unknown order: " + orderId);
```

```
        }
        return orders[orderId - 1];
    }
}
```

File `Main.java`:

```
public class Main {
    public static void main(String[] args) throws ShopException {
        Shop shop = new Shop();
        shop.addCustomer("Rossi Giovanni", "greds@torino.it");
        shop.addCustomer("Bianchi Anna", "anna@polito.it");
        try {
            shop.addCustomer("G. Rossi", "GREDS@Torino.it");
        } catch (ShopException e) {
            IO.println("error: " + e.getMessage());
        }
        Customer x = new Customer("X", "anna@POLITO.it");
        Customer y = new Customer("Anna B.", "anna@polito.it");
        IO.println(x.equals(y) + " " + (x.hashCode() == y.hashCode()) + " " + x.equals(null) + " "
                + x.equals("anna@polito.it"));
        IO.println(Shop.countDistinct(x, y, new Customer("Z", "z@z.it"),
                "anna@polito.it", "anna@polito.it"));
        int o1 = shop.newOrder("greds@torino.it");
        shop.addItem(o1, "Pizza", 2);
        shop.addItem(o1, "Cola", 1);
        int o2 = shop.repeat(o1);
        shop.addItem(o2, "Pizza", 1);
        shop.addItem(o2, "Tiramisu", 3);
        IO.println(shop.show(o1));
        IO.println(shop.show(o2));
    }
}
```

`equals` takes an `Object` (an `equals(Customer)` would be an overload that `countDistinct(Object...)` never calls) and uses `instanceof`, which is false for `null` and for a `String`; both `equals` and `hashCode` are computed from the same key, the email in lower case with `Locale.ROOT`, so equal customers always get the same hash (with a Turkish default locale `"I".toLowerCase()` is `"ı"`, which would break the pair `equalsIgnoreCase`/`toLowerCase()`). Because equality depends only on the email, `newOrder` finds the registered customer with a probe `new Customer("", email)`, and the order keeps the registered object, so it shows `greds@torino.it` whatever case was typed. `clone()` calls `super.clone()`, which copies the fields one by one (the copy would still share the two arrays), then clones `items` and `qty`; the customer can stay shared because it is immutable, and the `CloneNotSupportedException` cannot happen, so it becomes an `AssertionError`. `countDistinct` counts an element only if no earlier element is equal to it, with `Objects.equals` so that `null` arguments do not crash.

5.3 Bank operations: abstract classes and Template Method

Package `ex5_3`.

File `LedgerException.java`:

```
public class LedgerException extends Exception {
    public LedgerException(String message) {
        super(message);
    }
}
```

File `ByAmountDesc.java`:

```
public class ByAmountDesc extends Sorter {

    @Override
    protected int compare(Object a, Object b) {
        Operation x = (Operation) a;
        Operation y = (Operation) b;
        // not (int) (y - x): 0.5 would become 0
        return Double.compare(y.getAmount(), x.getAmount());
    }
}
```

File `ByDateDesc.java`:

```
public class ByDateDesc extends Sorter {

    @Override
    protected int compare(Object a, Object b) {
        Operation x = (Operation) a;
        Operation y = (Operation) b;
        return Integer.compare(y.getDate(), x.getDate());   // y before x: decreasing
    }
}
```

File `Deposit.java`:

```
public class Deposit extends Operation {

    public Deposit(int date, double amount) {
        super(date, amount);
    }

    @Override
    protected char sign() {
        return '+';
    }
}
```

File `Operation.java`:

```java
import java.util.Locale;

public abstract class Operation {

    private final int date;        // day of the year, 1-365
    private final double amount;   // always positive: the sign says which way it goes

    protected Operation(int date, double amount) {   // protected: only subclasses call it
        this.date = date;
        this.amount = amount;
    }

    public int getDate() { return date; }

    public double getAmount() { return amount; }

    // the only step that changes between subclasses
    protected abstract char sign();

    // the template: final, so no subclass can change the format, only its sign
    @Override
    public final String toString() {
        return String.format(Locale.US, "%d,%.2f%c", date, amount, sign());
    }

    public final double effect() {
        return sign() == '-' ? -amount : amount;
    }
}
```

File `Sorter.java`:

```java
public abstract class Sorter {

    // the algorithm is written once; the subclasses only say how two elements compare
    public void sort(Object[] v) {
        for (int pass = 0; pass < v.length - 1; pass++) {
            // the last 'pass' cells are already in place
            for (int i = 0; i < v.length - 1 - pass; i++) {
                // > 0, not >= 0: equal elements are never swapped
                if (compare(v[i], v[i + 1]) > 0) {
                    Object t = v[i];
                    v[i] = v[i + 1];
                    v[i + 1] = t;
                }
            }
        }
    }

    /** Negative if a comes first, positive if b comes first, 0 if equal for this order. */
    protected abstract int compare(Object a, Object b);
}
```

File `Withdrawal.java`:

```java
public class Withdrawal extends Operation {

    public Withdrawal(int date, double amount) {
        super(date, amount);
    }

    @Override
    protected char sign() {
        return '-';
    }
}
```

File `Ledger.java`:

```java
import java.util.Arrays;

public class Ledger {

    private static final int MAX_OPERATIONS = 100;

    private final String owner;
    // Deposit and Withdrawal objects together
    private final Operation[] ops = new Operation[MAX_OPERATIONS];
    private int count = 0;

    public Ledger(String owner, int date, double initial) {
        this.owner = owner;
        add(new Deposit(date, initial));   // opening the ledger is the first deposit
    }

    public String getOwner() { return owner; }

    public void deposit(int date, double amount) {
        add(new Deposit(validDate(date), amount));
    }

    public void withdraw(int date, double amount) throws LedgerException {
        if (amount > balance()) {
            throw new LedgerException("insufficient funds");
        }
        add(new Withdrawal(validDate(date), amount));
    }

    public double balance() {
        double b = 0;
        for (int i = 0; i < count; i++) {
            b += ops[i].effect();   // no instanceof: each operation knows its sign
        }
        return b;
    }

    public String[] movements() {
        return sorted(Arrays.copyOf(ops, count), new ByDateDesc());
    }

    public String[] deposits() {
        return sorted(select(true), new ByAmountDesc());
    }

    public String[] withdrawals() {
        return sorted(select(false), new ByAmountDesc());
```

```
    }

    // as at the exam: an operation cannot be dated before the last one
    private int validDate(int date) {
        return Math.max(date, ops[count - 1].getDate());
    }

    private void add(Operation op) {
        if (count == ops.length) {
            throw new IllegalStateException("full");
        }
        ops[count++] = op;
    }

    private Operation[] select(boolean deposits) {
        Operation[] result = new Operation[count];
        int n = 0;
        for (int i = 0; i < count; i++) {
            if ((ops[i] instanceof Deposit) == deposits) {
                result[n++] = ops[i];
            }
        }
        return Arrays.copyOf(result, n);
    }

    // sorts a copy, never the ledger's own array, then turns the operations into text
    private static String[] sorted(Operation[] v, Sorter sorter) {
        sorter.sort(v);   // an Operation[] is an Object[]
        String[] s = new String[v.length];
        for (int i = 0; i < v.length; i++) {
            s[i] = v[i].toString();
        }
        return s;
    }
}
```

File `Main.java`:

```
import java.util.Locale;

public class Main {
    public static void main(String[] args) throws LedgerException {
        Ledger l = new Ledger("Paul", 1, 500);
        l.deposit(10, 250.5);
        l.withdraw(5, 100);                 // day 5 is before day 10: done on day 10
        l.withdraw(20, 40.25);
        try {
            l.withdraw(21, 1000);
        } catch (LedgerException e) {
            IO.println("error: " + e.getMessage());
        }
        IO.println(String.join(" ", l.movements()));
        IO.println(String.join(" ", l.deposits()) + " | " + String.join(" ", l.withdrawals()));
        IO.println(String.format(Locale.US, "%.2f", l.balance()));
        Operation op = new Deposit(3, 7);
        IO.println(op + " " + op.effect() + " " + new Withdrawal(4, 2.5).effect());
    }
}
```

`Operation` is a Template Method: the final `toString()` and `effect()` fix the format and the arithmetic, and the subclasses only supply `sign()`, so `Deposit` and `Withdrawal` are three lines each and `balance()` is a sum of `effect()` with no `instanceof`. The `Sorter` works on `Object[]`, so the comparators cast to `Operation` and use `Integer.compare`/`Double.compare` with the arguments swapped for a decreasing order; a subtraction cast to `int` would turn a difference of 0.5 into 0. The bubble sort swaps only when `compare` is strictly positive, which makes it stable: the two operations of day 10 stay in insertion order in `movements()`. The ledger never sorts its own array: each report sorts a copy (`Arrays.copyOf` or a filtered array), otherwise `movements()` would reorder the history that `validDate` relies on, since it reads the last operation added. `withdraw` checks the funds before creating the operation, so a refused withdrawal leaves no trace. The amounts are `double`s compared with no tolerance, so the same trap as in 5.1 applies ($0.3 - 0.1$ is a little less than 0.2).

5.4 Pricing rules: interfaces, static and default methods

Package `ex5_4`.

File `CheckoutException.java`:

```
public class CheckoutException extends Exception {
    public CheckoutException(String message) {
        super(message);
    }
}
```

File `Member.java`:

```
public interface Member {
    default String badge() { return "member"; }
}
```

File `Pricing.java`:

```java
public interface Pricing {

    // the only abstract method: a lambda or an anonymous class can implement it
    double apply(double price);

    default String describe() {
        return "custom";
    }

    default Pricing then(Pricing next) {
        return new Pricing() {
            @Override
            public double apply(double price) {
                // here plain 'this' is the anonymous object:
                // Pricing.this is the rule then() was called on
                return next.apply(Pricing.this.apply(price));
            }

            @Override
            public String describe() {
                return Pricing.this.describe() + " then " + next.describe();
            }
        };
    }

    // static factories: callers see only the interface, the classes stay package-private
    static Pricing none() {
        return new NoDiscount();
    }

    static Pricing percentOff(int percent) {
        if (percent < 0 || percent > 100) {
            throw new IllegalArgumentException("percent must be 0-100: " + percent);
        }
        return new PercentOff(percent);
    }

    static Pricing fixedOff(double amount) {
        return new FixedOff(amount);
    }
}
```

File `Student.java`:

```java
public interface Student {
    default String badge() { return "student"; }
}
```

File `FixedOff.java`:

```java
import java.util.Locale;

// package-private: created only through Pricing.fixedOff()
class FixedOff implements Pricing {

    private final double amount;

    FixedOff(double amount) {
        this.amount = amount;
    }

    @Override
    public double apply(double price) {
        return Math.max(0, price - amount);   // never below 0
    }

    @Override
    public String describe() {
        return String.format(Locale.US, "-%.2f", amount);
    }
}
```

File `NoDiscount.java`:

```java
// package-private: created only through Pricing.none()
class NoDiscount implements Pricing {

    @Override
    public double apply(double price) {
        return price;
    }

    @Override
    public String describe() {
        return "none";
    }
}
```

File `PercentOff.java`:

```java
// package-private: created only through Pricing.percentOff(), which checks the range
class PercentOff implements Pricing {

    private final int percent;

    PercentOff(int percent) {
        this.percent = percent;
    }

    @Override
    public double apply(double price) {
        return price * (100 - percent) / 100;
    }

    @Override
    public String describe() {
        return "-" + percent + "%";
    }
}
```

File `StudentMember.java`:

```
public class StudentMember implements Member, Student {

    // two inherited defaults with the same signature:
    // the class must override, or it does not compile
    @Override
    public String badge() {
        // Interface.super picks one version
        return Member.super.badge() + "+" + Student.super.badge();
    }
}
```

File `Checkout.java`:

```
import java.util.Locale;

public class Checkout {

    private static final int MAX_PRODUCTS = 50;

    // parallel arrays: cell i of each one describes product i
    private final String[] codes = new String[MAX_PRODUCTS];
    private final double[] prices = new double[MAX_PRODUCTS];
    private final Pricing[] rules = new Pricing[MAX_PRODUCTS];
    private int count = 0;

    public void addProduct(String code, double price) throws CheckoutException {
        if (indexOf(code) >= 0) {
            throw new CheckoutException("duplicate product " + code);
        }
        if (count == codes.length) {
            throw new IllegalStateException("full");
        }
        codes[count] = code;
        prices[count] = price;
        rules[count] = Pricing.none();   // the default rule: no null checks later
        count++;
    }

    public void setRule(String code, Pricing rule) throws CheckoutException {
        rules[find(code)] = rule;
    }

    public double price(String code) throws CheckoutException {
        int i = find(code);
        return Math.round(rules[i].apply(prices[i]) * 100) / 100.0;   // to the cent
    }

    public String describe(String code) throws CheckoutException {
        int i = find(code);
        return String.format(Locale.US, "%s %.2f %s = %.2f",
                code, prices[i], rules[i].describe(), price(code));
    }

    private int find(String code) throws CheckoutException {
        int i = indexOf(code);
        if (i < 0) {
            throw new CheckoutException("unknown product " + code);
        }
        return i;
    }
```

```
    private int indexOf(String code) {
        for (int i = 0; i < count; i++) {
            if (codes[i].equals(code)) {
                return i;
            }
        }
        return -1;
    }
}
```

File `Main.java`:

```
public class Main {
    public static void main(String[] args) throws CheckoutException {
        Checkout c = new Checkout();
        c.addProduct("BOOK", 20);
        c.addProduct("PEN", 1.99);
        c.addProduct("BAG", 3);
        c.addProduct("CAP", 9.5);
        c.setRule("BOOK", Pricing.percentOff(10).then(Pricing.fixedOff(5)));
        c.setRule("PEN", Pricing.percentOff(33));
        c.setRule("BAG", Pricing.fixedOff(5));
        Pricing half = new Pricing() {          // an anonymous class
            @Override
            public double apply(double price) { return price / 2; }
        };
        for (String code : new String[]{"BOOK", "PEN", "BAG", "CAP"}) {
            IO.println(c.describe(code));
        }
        IO.println(half.apply(9) + " " + half.describe() + " " + Pricing.none().apply(4.5) + " "
                + Pricing.none().describe());
        IO.println(new StudentMember().badge());
        try {
            Pricing.percentOff(120);
        } catch (IllegalArgumentException e) {
            IO.println("error: " + e.getMessage());
        }
    }
}
```

`Pricing` has one abstract method, so any rule can be an anonymous class or a lambda, and a rule that does not override `describe()` is `custom`; `then` returns an anonymous class in which plain `this` is the new object, so the first rule is reached with `Pricing.this` (and `next` is captured, being effectively final). The static factories return objects of package-private classes (`NoDiscount`, `PercentOff`, `FixedOff`), so outside the package a caller sees only the interface, and `percentOff` validates the range before creating anything. `StudentMember` inherits two `badge()` defaults with the same signature, so it does not compile until it overrides the method, and inside it `Member.super.badge()` and `Student.super.badge()` select the two versions. `Checkout` gives every product `Pricing.none()` when it is added, so `price` never checks for `null`, and rounds once at the end with `Math.round(x * 100) / 100.0`. The text asks for a `CheckoutException` on a duplicate code but gives no message; this solution uses `duplicate product <code>`, and `unknown product <code>` for an unknown code. One trap of doubles remains: `1.005` is stored as 1.00499..., so `Math.round(1.005 * 100) / 100.0` gives 1.00, while `%.2f` on `1.005` prints `1.01`; with prices of two decimals, as in the exams, it never shows.

5.5 Menu combos: Composite

Package `ex5_5`.

File `MenuException.java`:

```
public class MenuException extends Exception {
    public MenuException(String message) {
        super(message);
    }
}
```

File `Composed.java`:

```
// A flag (marker) interface: no methods, it only says "this entry is made of other entries".
public interface Composed {
}
```

File `Dish.java`:

```
// The Leaf. The accessors name() and price() generated by the record implement the interface.
public record Dish(String name, double price) implements MenuEntry {

    @Override
    public String describe() {
        return name;
    }
}
```

File `MenuEntry.java`:

```
// The Component of the Composite pattern: dishes and combos are used through this type only.
public interface MenuEntry {

    String name();

    double price();

    String describe();

    // parentheses only around composite parts, so that nesting stays readable
    static String wrap(MenuEntry e) {
        if (e instanceof Composed) {
            return "(" + e.describe() + ")";
        }
        return e.describe();
    }
}
```

File `Combo.java`:

```
// The Composite: its parts are MenuEntry objects, so a combo can contain other combos.
public class Combo implements MenuEntry, Composed {
    private final String name;
    private final int discountPercent;
    private final MenuEntry[] parts;

    public Combo(String name, int discountPercent, MenuEntry... parts) {
        if (parts == null || parts.length == 0) {
            throw new IllegalArgumentException("a combo needs at least one part");
        }
        if (discountPercent < 0 || discountPercent > 50) {
            throw new IllegalArgumentException("discount must be 0-50: " + discountPercent);
        }
        this.name = name;
```

```
        this.discountPercent = discountPercent;
        this.parts = parts.clone();     // the caller may pass an array and change it later
    }

    @Override
    public String name() {
        return name;
    }

    @Override
    public double price() {
        double sum = 0;
        for (MenuEntry part : parts) {
            sum += part.price();        // dynamic binding: a Dish or a nested Combo
        }
        return Math.round(sum * (100 - discountPercent)) / 100.0;
    }

    @Override
    public String describe() {
        String[] texts = new String[parts.length];
        for (int i = 0; i < parts.length; i++) {
            texts[i] = MenuEntry.wrap(parts[i]);
        }
        return name + ": " + String.join(" + ", texts);
    }

    public MenuEntry[] parts() {
        return parts.clone();
    }
}
```

File `Menu.java`:

```
import java.util.Locale;

public class Menu {
    public static final int MAX_ENTRIES = 50;   // the capacity given in the text

    private final MenuEntry[] entries = new MenuEntry[MAX_ENTRIES];
    private int count = 0;

    public void add(MenuEntry e) throws MenuException {
        for (int i = 0; i < count; i++) {
            if (entries[i].name().equals(e.name())) {
                throw new MenuException("duplicate entry " + e.name());
            }
        }
        if (count == entries.length) {
            throw new IllegalStateException("full");
        }
        entries[count++] = e;
    }

    public String card() {
        String card = "";
        for (int i = 0; i < count; i++) {
            card += String.format(Locale.US, "%-8s%6.2f\n", entries[i].name(), entries[i].price());
        }
        return card;
    }
```

```
    // the recursion follows the structure: a combo has as many dishes as its parts together
    public static int countDishes(MenuEntry e) {
        if (e instanceof Combo c) {
            int n = 0;
            for (MenuEntry part : c.parts()) {
                n += countDishes(part);
            }
            return n;
        }
        return e instanceof Dish ? 1 : 0;
    }
}
```

File `Main.java`:

```
public class Main {
    public static void main(String[] args) throws MenuException {
        MenuEntry burger = new Dish("Burger", 8), fries = new Dish("Fries", 3);
        MenuEntry drinks = new Combo("Drinks", 10, new Dish("Cola", 2.5), new Dish("Water", 1.5));
        MenuEntry lunch = new Combo("Lunch", 20, burger, fries, drinks);
        Menu menu = new Menu();
        menu.add(burger);
        menu.add(fries);
        menu.add(drinks);
        menu.add(lunch);
        IO.println(drinks.describe() + " = " + drinks.price());
        IO.println(lunch.describe() + " = " + lunch.price());
        IO.println(Menu.countDishes(lunch) + " " + Menu.countDishes(fries) + " "
                + (lunch instanceof Composed) + " " + (burger instanceof Composed));
        IO.print(menu.card());
        try {
            menu.add(new Dish("Lunch", 1));
        } catch (MenuException e) {
            IO.println("error: " + e.getMessage());
        }
    }
}
```

`MenuEntry` is the Component of the Composite pattern: `Combo` keeps its parts as `MenuEntry` objects, so `price()` and `describe()` call the parts without knowing whether they are dishes or nested combos, and dynamic binding does the recursion. The empty `Composed` interface is only a flag tested with `instanceof` in `MenuEntry.wrap`, while `countDishes` needs the concrete `Combo` (pattern-matching `instanceof`), because a flag has no methods to reach the parts. The trap in the price is the order of the rounding: apply the discount to the sum and round once, with exactly `Math.round(sum * (100 - discount)) / 100.0`, so that 14.60 at 20% gives 11.68. The constructor and `parts()` both clone the array, otherwise the caller could change a combo from outside through the varargs array or the returned one. `Menu` holds the 50 entries of the text, duplicates are found by name, and `card()` needs `Locale.US` and `\n` exactly as in the format string.

5.6 Restaurant chain reports: Comparable, Comparator, Strategy

Package `ex5_6`.

File `ChainException.java`:

```
public class ChainException extends Exception {
    public ChainException(String message) {
        super(message);
    }
}
```

File `ByIncomeDesc.java`:

```
import java.util.Comparator;

// Decreasing income, ties by name. Double.compare, never a subtraction: a difference such as
// 0.4 would be truncated to 0 by an (int) cast, and with ints a subtraction can overflow.
public class ByIncomeDesc implements Comparator<Restaurant> {

    @Override
    public int compare(Restaurant a, Restaurant b) {
        int c = Double.compare(b.getIncome(), a.getIncome());   // b first: decreasing
        if (c != 0) {
            return c;
        }
        return a.compareTo(b);
    }
}
```

File `Restaurant.java`:

```
public class Restaurant implements Comparable<Restaurant> {
    private final String name;
    private final int tables;
    private int usedTables = 0;
    private int refused = 0;
    private double income = 0;

    public Restaurant(String name, int tables) {
        this.name = name;
        this.tables = tables;
    }

    // tables of four seats: a group of people needs ceil(people / 4) of them
    public void book(int people) {
        int needed = (people + 3) / 4;
        if (needed <= getUnusedTables()) {
            usedTables += needed;
        } else {
            refused += people;
        }
    }

    public void pay(double amount) {
        income += amount;
    }

    public String getName() {
        return name;
    }

    public double getIncome() {
        return income;
    }
```

```
    public int getRefused() {
        return refused;
    }

    public int getUnusedTables() {
        return tables - usedTables;
    }

    // natural order: by name
    @Override
    public int compareTo(Restaurant other) {
        return name.compareTo(other.name);
    }
}
```

File `Chain.java`:

```
import java.util.Arrays;
import java.util.Comparator;

public class Chain {
    public static final int MAX_RESTAURANTS = 20;

    private final Restaurant[] restaurants = new Restaurant[MAX_RESTAURANTS];
    private int count = 0;

    public void addRestaurant(String name, int tables) throws ChainException {
        if (find(name) != null) {
            throw new ChainException("duplicate restaurant " + name);
        }
        if (count == restaurants.length) {
            throw new IllegalStateException("full");
        }
        restaurants[count++] = new Restaurant(name, tables);
    }

    public Restaurant getRestaurant(String name) throws ChainException {
        Restaurant r = find(name);
        if (r == null) {
            throw new ChainException("unknown restaurant " + name);
        }
        return r;
    }

    public String[] byName() {
        Restaurant[] v = copy();
        Arrays.sort(v);                 // no comparator: the natural order, compareTo
        return names(v);
    }

    public String[] sortByIncome() {
        return sorted(new ByIncomeDesc());
    }

    public String[] sortByRefused() {
        return sorted(new Comparator<Restaurant>() {
            @Override
            public int compare(Restaurant a, Restaurant b) {
                int c = Integer.compare(a.getRefused(), b.getRefused());
                if (c != 0) {
                    return c;
                }
```

```
                return a.compareTo(b);
            }
        });
    }

    public String[] sortByUnusedTables() {
        return sorted((a, b) -> {
            int c = Integer.compare(a.getUnusedTables(), b.getUnusedTables());
            if (c != 0) {
                return c;
            }
            return a.compareTo(b);
        });
    }

    // Strategy: the caller chooses the order. Arrays.sort on objects is stable, and copy()
    // keeps the insertion order, so restaurants that compare as equal stay in that order.
    public String[] top(int n, Comparator<Restaurant> order) {
        if (n < 0) {
            throw new IllegalArgumentException("n must not be negative: " + n);
        }
        return Arrays.copyOf(sorted(order), Math.min(n, count));
    }

    private String[] sorted(Comparator<Restaurant> order) {
        Restaurant[] v = copy();
        Arrays.sort(v, order);
        return names(v);
    }

    // sorting a copy leaves the insertion order of the chain untouched
    private Restaurant[] copy() {
        return Arrays.copyOf(restaurants, count);
    }

    private static String[] names(Restaurant[] v) {
        String[] names = new String[v.length];
        for (int i = 0; i < v.length; i++) {
            names[i] = v[i].getName();
        }
        return names;
    }

    private Restaurant find(String name) {
        for (int i = 0; i < count; i++) {
            if (restaurants[i].getName().equals(name)) {
                return restaurants[i];
            }
        }
        return null;
    }
}
```

File `Main.java`:

```
import java.util.Arrays;
import java.util.Comparator;

public class Main {
    public static void main(String[] args) throws ChainException {
        Chain ch = new Chain();
        ch.addRestaurant("Gino", 3);
        ch.addRestaurant("Arturo", 2);
        ch.addRestaurant("Bice", 5);
        ch.addRestaurant("Dora", 2);
        ch.getRestaurant("Gino").book(10);
        ch.getRestaurant("Gino").book(2);
        ch.getRestaurant("Arturo").book(9);
        ch.getRestaurant("Bice").book(4);
        ch.getRestaurant("Dora").book(8);
        ch.getRestaurant("Gino").pay(120.5);
        ch.getRestaurant("Bice").pay(80);
        ch.getRestaurant("Dora").pay(120.5);
        IO.println(Arrays.toString(ch.byName()));
        IO.println(Arrays.toString(ch.sortByIncome()));
        IO.println(Arrays.toString(ch.sortByRefused()));
        IO.println(Arrays.toString(ch.sortByUnusedTables()));
        Comparator<Restaurant> byNameDesc = Comparator.comparing(Restaurant::getName).reversed();
        IO.println(Arrays.toString(ch.top(2, byNameDesc)));
        IO.println(Arrays.toString(ch.top(3, Comparator.comparingDouble(Restaurant::getIncome))));
        try {
            ch.getRestaurant("Zio");
        } catch (ChainException e) {
            IO.println("error: " + e.getMessage());
        }
    }
}
```

The four reports share one private helper that sorts a copy of the array (so the chain keeps its insertion order) and extracts the names; only the comparator changes, which is exactly the Strategy pattern that `top` then offers to the caller. The same ordering rule is written three ways, as required: a named class `ByIncomeDesc`, an anonymous class and a lambda, and all three break ties with the natural order, `a.compareTo(b)`. No comparator subtracts: with doubles, `(int) (b.getIncome() - a.getIncome())` turns 0.4 into 0, and with ints a subtraction can overflow, so they use `Double.compare` and `Integer.compare`, swapping the arguments for a decreasing order. `top` relies on `Arrays.sort` being stable for objects: Gino and Dora both earn 120.50, and Gino comes first only because it was added first. A group that needs $\lceil \text{people} / 4 \rceil$ tables is `(people + 3) / 4` in integer arithmetic.

5.7 A train timetable: Iterable and Iterator

Package `ex5_7`.

File `Train.java`:

```
import java.util.Locale;

// departure: minutes after midnight
public record Train(String code, int departure, String destination) {

    @Override
    public String toString() {
        return String.format(Locale.US, "%02d:%02d %s %s",
                departure / 60, departure % 60, code, destination);
    }
}
```

File `Timetable.java`:

```
import java.util.Iterator;
import java.util.NoSuchElementException;

public class Timetable implements Iterable<Train> {

    // static: a node does not need to know the timetable it belongs to
    private static class Node {
        private final Train train;
        private Node next;

        Node(Train train, Node next) {
            this.train = train;
            this.next = next;
        }
    }

    private Node head = null;
    private int size = 0;

    // keeps the list sorted by departure; the new train goes AFTER the ones with the same
    // departure (<=, not <), so equal departures stay in insertion order
    public void add(Train t) {
        if (head == null || t.departure() < head.train.departure()) {
            head = new Node(t, head);
        } else {
            Node prev = head;
            while (prev.next != null && prev.next.train.departure() <= t.departure()) {
                prev = prev.next;
            }
            prev.next = new Node(t, prev.next);
        }
        size++;
    }

    public int size() {
        return size;
    }

    @Override
    public Iterator<Train> iterator() {
        return new TrainIterator();
    }

    // inner (not static): it reads the field head of the timetable that created it
    private class TrainIterator implements Iterator<Train> {
        private Node current = head;
```

```
        @Override
        public boolean hasNext() {
            return current != null;
        }

        @Override
        public Train next() {
            if (current == null) {
                throw new NoSuchElementException();
            }
            Train t = current.train;
            current = current.next;
            return t;
        }
    }

    // A view, not a copy: nothing is collected here. The list is read only when somebody
    // iterates, so trains added later appear too. from and to are captured by the anonymous
    // classes, so they must be effectively final.
    public Iterable<Train> between(int from, int to) {
        return new Iterable<>() {
            @Override
            public Iterator<Train> iterator() {
                return new Iterator<>() {
                    private Node current = head;

                    @Override
                    public boolean hasNext() {
                        // skip the trains outside [from, to)
                        while (current != null && (current.train.departure() < from
                                || current.train.departure() >= to)) {
                            current = current.next;
                        }
                        return current != null;
                    }

                    @Override
                    public Train next() {
                        if (!hasNext()) {
                            throw new NoSuchElementException();
                        }
                        Train t = current.train;
                        current = current.next;
                        return t;
                    }
                };
            }
        };
    }

    public Iterable<Train> to(String destination) {
        return new Iterable<>() {
            @Override
            public Iterator<Train> iterator() {
                return new Iterator<>() {
                    private Node current = head;

                    @Override
                    public boolean hasNext() {
                        while (current != null
                                && !current.train.destination().equalsIgnoreCase(destination)) {
                            current = current.next;
                        }
```

```
                    return current != null;
                }

                @Override
                public Train next() {
                    if (!hasNext()) {
                        throw new NoSuchElementException();
                    }
                    Train t = current.train;
                    current = current.next;
                    return t;
                }
            };
        }
    };
}

// the for-each on this works because Timetable is Iterable; the list is sorted,
// so the first train at minute or later is the answer
public Train next(int minute) {
    for (Train t : this) {
        if (t.departure() >= minute) {
            return t;
        }
    }
    return null;
}
}
```

File `Main.java`:

```
import java.util.Iterator;
import java.util.NoSuchElementException;

public class Main {
    public static void main(String[] args) {
        Timetable t = new Timetable();
        t.add(new Train("RV2021", 9 * 60 + 5, "Aosta"));
        t.add(new Train("FR9611", 8 * 60 + 15, "Milano"));
        t.add(new Train("R10023", 9 * 60 + 5, "Chivasso"));
        t.add(new Train("FR9635", 17 * 60 + 15, "Milano"));
        for (Train tr : t) {
            IO.println(tr);
        }
        String s = "";
        for (Train tr : t.between(9 * 60, 12 * 60)) {
            s += tr.code() + " ";
        }
        IO.println(s.trim());
        s = "";
        for (Train tr : t.to("MILANO")) {
            s += tr.code() + " ";
        }
        IO.println(s.trim());
        IO.println(t.next(10 * 60) + " | " + t.next(18 * 60));
        Iterator<Train> it = t.iterator();
        for (int i = 0; i < t.size(); i++) {
            it.next();
        }
        try {
            it.next();
        } catch (NoSuchElementException e) {
```

```
            IO.println("no more trains");
        }
    }
}
```

The list node is a private static nested class, because a node does not need its timetable, while the iterator is a private inner class, because it reads the field `head` of the timetable that created it. To keep equal departures in insertion order, `add` walks past every node with departure `<=` the new one and inserts after them; using `<` would put the newer train first. `between` and `to` are views: they return an anonymous `Iterable` whose anonymous iterator starts from `head` only when someone iterates, so a view taken before an `add` also shows the new train, and the captured parameters (`from`, `to`, `destination`) must be effectively final. Their `hasNext()` skips the trains that do not match and can be called many times without consuming anything, and every `next()` throws `NoSuchElementException` at the end, as the `Iterator` contract requires. `next(int)` is a for-each on `this`, which compiles only because `Timetable` implements `Iterable<Train>`, and it can stop at the first match because the list is sorted.

5.8 Order statistics: functional interfaces, lambdas, method references

Package `ex5_8`.

File `Order.java`:

```
public record Order(int id, String customer, double total, boolean delivered) {
}
```

File `Stats.java`:

```
import java.util.Comparator;
import java.util.function.Consumer;
import java.util.function.Function;
import java.util.function.IntFunction;
import java.util.function.Predicate;
import java.util.function.ToDoubleFunction;

// Each method fixes the loop and receives the varying part (the behaviour) as an object
// of a functional interface: a lambda or a method reference at the call site.
public class Stats {

    private Stats() {       // only static methods: no instances
    }

    public static int count(Order[] v, Predicate<Order> cond) {
        int n = 0;
        for (Order o : v) {
            if (cond.test(o)) {
                n++;
            }
        }
        return n;
    }

    // ToDoubleFunction returns a primitive double: no boxing, unlike Function<Order, Double>
    public static double sum(Order[] v, ToDoubleFunction<Order> value) {
        double sum = 0;
        for (Order o : v) {
            sum += value.applyAsDouble(o);
        }
```

```
        return sum;
    }

    // on ties the first maximum wins (> 0, not >= 0)
    public static Order max(Order[] v, Comparator<Order> order) {
        Order best = null;
        for (Order o : v) {
            if (best == null || order.compare(o, best) > 0) {
                best = o;
            }
        }
        return best;
    }

    public static void forEachIf(Order[] v, Predicate<Order> cond, Consumer<Order> action) {
        for (Order o : v) {
            if (cond.test(o)) {
                action.accept(o);
            }
        }
    }

    public static String[] map(Order[] v, Function<Order, String> f) {
        String[] result = new String[v.length];
        for (int i = 0; i < v.length; i++) {
            result[i] = f.apply(v[i]);
        }
        return result;
    }

    public static Order[] generate(int n, IntFunction<Order> maker) {
        Order[] result = new Order[n];
        for (int i = 0; i < n; i++) {
            result[i] = maker.apply(i);
        }
        return result;
    }
}
```

File `Main.java`:

```
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        Order[] orders = {
            new Order(1, "Rossi", 29.5, true),
            new Order(2, "Bianchi", 12.0, false),
            new Order(3, "Rossi", 8.0, true),
            new Order(4, "Verdi", 41.25, true)
        };
        IO.println(Stats.count(orders, o -> o.delivered()) + " "          // ->
                + Stats.count(orders, o -> o.total() > 20));
        IO.println(Stats.sum(orders, o -> o.total()));                    // ->
        IO.println(Stats.max(orders, (a, b) -> Double.compare(a.total(), b.total())).id());
        Stats.forEachIf(orders, o -> o.customer().equals("Rossi"),
                o -> IO.println(o));                                      // ->
        IO.println(Arrays.toString(Stats.map(orders, o -> o.customer())));  // ->
        IO.println(Arrays.toString(Stats.map(orders, o -> String.valueOf(o.id()))));
        Order[] gen = Stats.generate(3, i -> new Order(i + 1, "C" + i, 10 * (i + 1), false));
        IO.println(gen[2]);
    }
}
```

File `MainRefs.java`:

```
import java.util.Arrays;
import java.util.Comparator;

// The demo of Main after part 2. A lambda that only passes its parameter to one method becomes
// a method reference (Order::total is o -> o.total(), IO::println is o -> IO.println(o)), and
// Comparator.comparingDouble builds the comparator from the key. o -> o.total() > 20 also
// compares the result, so it has no equivalent method reference and stays a lambda.
public class MainRefs {
    public static void main(String[] args) {
        Order[] orders = {
            new Order(1, "Rossi", 29.5, true),
            new Order(2, "Bianchi", 12.0, false),
            new Order(3, "Rossi", 8.0, true),
            new Order(4, "Verdi", 41.25, true)
        };
        IO.println(Stats.count(orders, Order::delivered) + " "            // ->
                + Stats.count(orders, o -> o.total() > 20));
        IO.println(Stats.sum(orders, Order::total));                      // ->
        IO.println(Stats.max(orders, Comparator.comparingDouble(Order::total)).id());
        Stats.forEachIf(orders, o -> o.customer().equals("Rossi"),
                IO::println);                                             // ->
        IO.println(Arrays.toString(Stats.map(orders, Order::customer)));  // ->
        IO.println(Arrays.toString(Stats.map(orders, o -> String.valueOf(o.id()))));
        Order[] gen = Stats.generate(3, i -> new Order(i + 1, "C" + i, 10 * (i + 1), false));
        IO.println(gen[2]);
    }
}
```

Every method of `Stats` writes the loop once and receives the part that varies as a functional interface from `java.util.function`, so the caller supplies it as a lambda or a method reference; `ToDoubleFunction` is used instead of `Function<Order, Double>` to avoid boxing. `max` returns `null` for an empty array and keeps the first maximum on ties, because it replaces the best only when `compare` is strictly positive. In `MainRefs` a lambda becomes a method reference only when it just passes its parameter to one method: `o -> o.total()` is `Order::total` and `o -> IO.println(o)` is `IO::println`, while `o -> o.total() > 20` (on a marked line) also compares the result and must stay a lambda. `Comparator.comparingDouble(Order::total)` builds the same comparator as `(a, b) -> Double.compare(a.total(), b.total())`, so the two mains print the same lines.

5.9 Delivery notifications: Observer by composition (code + test)

Package `ex5_9`.

File `Status.java`:

```
public enum Status { NEW, CONFIRMED, ON_DELIVERY, DELIVERED }
```

File `Dispatcher.java`:

```
import java.beans.PropertyChangeEvent;
import java.beans.PropertyChangeListener;

public class Dispatcher implements PropertyChangeListener {

    @Override
    public void propertyChange(PropertyChangeEvent evt) {
        // the event carries its source, so one dispatcher can observe many orders
        if ("status".equals(evt.getPropertyName()) && evt.getNewValue() == Status.ON_DELIVERY) {
            Order o = (Order) evt.getSource();
            IO.println("Courier for order " + o.getId());
        }
    }
}
```

File `Order.java`:

```
import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;

// Observer by composition: Order HAS a PropertyChangeSupport and delegates to it, so it stays
// free to extend another class, and only the methods we choose are visible to the clients.
public class Order {
    private final int id;
    private Status status = Status.NEW;
    // this = the event source
    private final PropertyChangeSupport support = new PropertyChangeSupport(this);

    public Order(int id) {
        this.id = id;
    }

    public int getId() {
        return id;
    }

    public Status getStatus() {
        return status;
    }

    public void advance() {
        if (status == Status.DELIVERED) {
            // before any change or event
            throw new IllegalStateException("order " + id + " already delivered");
        }
        Status old = status;
        status = Status.values()[status.ordinal() + 1];
        // after the change: listeners see the new state
        support.firePropertyChange("status", old, status);
    }

    public void addListener(PropertyChangeListener l) {
        support.addPropertyChangeListener(l);
    }

    public void removeListener(PropertyChangeListener l) {
        support.removePropertyChangeListener(l);
    }
}
```

File `Main.java`:

```
import java.beans.PropertyChangeListener;

public class Main {
    public static void main(String[] args) {
        Order o = new Order(7);
        Dispatcher d = new Dispatcher();
        PropertyChangeListener log = evt -> IO.println("Order " + ((Order) evt.getSource()).getId()
                + ": " + evt.getOldValue() + " -> " + evt.getNewValue());
        o.addListener(log);
        o.addListener(d);
        o.advance();
        o.advance();
        o.removeListener(log);
        o.advance();
        try {
            o.advance();
        } catch (IllegalStateException e) {
            IO.println("error: " + e.getMessage());
        }
    }
}
```

The tests (`test/ex5_9`):

File `OrderTest.java`:

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertSame;
import static org.junit.jupiter.api.Assertions.assertThrows;

import java.beans.PropertyChangeListener;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

class OrderTest {
    // written by the recording listener; JUnit creates a new OrderTest for every test,
    // so each test starts from 0 and null
    private int events = 0;
    private Object lastNewValue = null;

    private Order order;
    private PropertyChangeListener recorder;

    @BeforeEach
    void setUp() {
        order = new Order(1);
        // a lambda can change fields (not local variables): it captures this, not the values
        recorder = evt -> {
            events++;
            lastNewValue = evt.getNewValue();
        };
        order.addListener(recorder);
    }

    @Test
    void oneAdvanceSendsExactlyOneEvent() {
        order.advance();
        assertEquals(1, events);
        assertSame(Status.CONFIRMED, lastNewValue);
        assertSame(Status.CONFIRMED, order.getStatus());
```

```
    }

    @Test
    void removedListenerReceivesNothing() {
        order.removeListener(recorder);
        order.advance();
        assertEquals(0, events);
        assertSame(Status.CONFIRMED, order.getStatus());   // the order still moved
    }

    @Test
    void advanceAfterDeliveredThrowsAndSendsNothing() {
        order.advance();
        order.advance();
        order.advance();
        assertEquals(3, events);
        assertSame(Status.DELIVERED, lastNewValue);
        IllegalStateException e = assertThrows(IllegalStateException.class, () -> order.advance());
        assertEquals("order 1 already delivered", e.getMessage());
        assertEquals(3, events);                            // no event for the failed advance
        assertSame(Status.DELIVERED, order.getStatus());
    }
}
```

`Order` uses `PropertyChangeSupport` by composition: it owns one, created with `new PropertyChangeSupport(this)` so that `evt.getSource()` is the order, and `addListener`/`removeListener` simply delegate, leaving `Order` free to extend another class. In `advance` the check for `DELIVERED` comes first, so a failed call changes nothing and fires nothing; otherwise the status moves to `Status.values()[status.ordinal() + 1]` and the event is fired after the change, with the old and the new value. Listeners are notified in the order they were added, which is why the log line comes before `Courier for order 7`. In `OrderTest` the recording lambda can increment `events` and store the last new value because they are fields of the test object (a lambda cannot modify local variables), and JUnit creates a new test object for every test, so each one starts from zero.

Slide errata

These slide lines do not compile as written, or are wrong or outdated in Java 25; each code row was checked with `javac` 25 (snippets in `slide-errata/` of the project). Code copied from the PDFs also brings typographic quotes and dashes (’, “, –), which are illegal characters in Java source.

Deck	The slide says	Correct Java
J02	`Counter` chain calls `decrement(7)`	the method is never defined (see 2.2)
J02	`String in = java.lang.IO.readLine();`	`IO.readln()`
J02	`public static int inverse(double n){ return 1 / n; }`	return type `double` (“possible lossy conversion from double to int”)
J02	`Car.countCountBuiltCars`	`Car.countBuiltCars`
J02	`public final static double 2PI;`	an identifier cannot start with a digit, and Math.acos(-1) is π, not 2π: put 2 * Math.acos(-1) in a field named `TWO_PI`
J02	record vs class: `class Point(){ ... }`	`class Point { ... }`
J02	`StackOfInt`: `private Element head` and `public int void pop()`	`private Element head;` and `public int pop()`

Deck	The slide says	Correct Java
J02	an inner class “cannot be instantiated from a static method / other classes”	both work given an outer object, even inside a static method: `c.new Incrementer(2)` (if the constructor is visible)
J02, J04	`new Integer(88)`, `new Character('b')`, `finalize()`	still compile but deprecated (`finalize` is marked for removal): use `Integer.valueOf` / autoboxing
J02a	`"Human".subString(2)`	`substring`
J02a	‘è’ has code point `U+00C8`	`U+00E8` (U+00C8 is ‘È’); its UTF-8 bytes C3 A8 are right
J02a	UTF-16 uses 16 bits for `U+0`–`U+D7FF`	also for `U+E000`–`U+FFFF`; only code points above U+FFFF take 32 bits
J03	`Stack.pop()`: `throw new StackException()`	missing `;`, and `StackException` must be declared
J03	`@BeforeClass` / `@AfterClass`	JUnit 4 names; JUnit 5 uses static `@BeforeAll` / `@AfterAll`
J03	`@Test(expected=PossibleException.class)`	JUnit 4 only; JUnit 5 uses `assertThrows`
J03	`assumeTrue("X not available", res.isAvailable())`	JUnit 5: `assumeTrue(res.isAvailable(), "X not available")`
J03	suite with `@RunWith(JUnitPlatform.class)` / `@RunWith(Suite.class)`	JUnit 5: `@Suite` + `@SelectClasses` (org.junit.platform.suite.api)
J04	`super(...)` must be the first statement	Java 25 (JEP 513) allows statements before `super(...)` if they do not use the object being built, e.g. argument checks; calling an instance method there is still an error
J04	`Operation`'s constructor is named `Addition`	`public Operation(char o, Expression l, Expression r)`
J04	`class Value implements Expression { @Override void value(){…} @Override void formula(){…} }`	`public double eval()` and `public String formula()`
J04	`Objects[3]= 42;`	`objects[3] = 42;`
J04	static methods in interfaces “can be overridden”	they are not inherited, so they cannot be overridden: “static methods cannot be annotated with @Override”
J04	comparators `return a.id – b.id;`	`Integer.compare(sa.id, sb.id)` (see 5.6)
J04	`printer.consume("Hello!")`	`Consumer`'s method is `accept`
J02a	Charset methods getDecoder() / getEncoder()	newDecoder() / newEncoder() (used in 3.4)
J01	jar cvfm my.jar manifest.txt	adds no classes: jar cvfm my.jar manifest.txt *.class (and the manifest must end with a newline)
J02	final static float PI = 3.14;	3.14 is a double: 3.14f, or declare PI as double (“possible lossy conversion from double to float”)
J02	Character toUpper(), toLower()	toUpperCase(), toLowerCase()
J02	Integer.valueof(i)	Integer.valueOf(i)
J02	Incrementer byOne = c.buildIncrementer(1); written outside Counter	Counter.Incrementer byOne = ...; also Counter c = new Counter() needs its ;
J02	records “since Java 17”	standard since Java 16 (JEP 395)
J02a	boolean equals(String s)	the real signature is equals(Object o) – see 5.2
J03	assertEquals(1.0, Math.cos(3.14), 0.01)	fails: cos(3.14) is about -1, so the expected value is -1.0

Deck	The slide says	Correct Java
J03	JUnit 5 assertions include assertThat() for Hamcrest	Jupiter's Assertions has no assertThat: use Hamcrest's org.hamcrest.MatcherAssert.assertThat
J04	float f; f = 4.7; // legal	illegal (double to float): f = 4.7f;
J04	class Employee{ String toString(){...} }	public String toString(): an override cannot reduce visibility
J04	abstract Sorter: an extra } before abstract int compare(...)	remove that }; if compare is made protected (as in 5.3), the subclasses' compare must be protected or public
J04	ComplexPolar: mod*cos(arg), mod*sin(arg)	Math.cos / Math.sin, or import static java.lang.Math.*
J04	Iterator it = seq.iterator() ... IO.println(el);	missing ; and the variable is element, not el
J04	Observer Subject: String prop="ini"; ... property = val;	prop = val;
J04	java.lang.Comparator, java.utils.Arrays	java.util.Comparator, java.util.Arrays